Tabular Foundation Models: A Systematic Survey

Alberto Archetti[a,*], Marina Mastroleo[a], Mattia Sabella[a], Cinzia Cappiello[a] and Matteo Matteucci[a]

[a]Department of Electronics, Information, and Bioengineering, Politecnico di Milano, Via Giuseppe Ponzio, 34, Milan, 20133, Italy

ARTICLE INFO

Keywords:
tabular data
foundation models
in-context learning
systematic review
prior-data fitted networks

ABSTRACT

Tabular data is among the most common modalities in applied machine learning, yet tabular prediction has long lacked pretrained models: for two decades, gradient-boosted decision trees, trained from scratch on each new dataset, have remained unmatched. Tabular foundation models are beginning to change this. A single network, pretrained once on a broad distribution of datasets, adapts to an unseen dataset with little dataset-specific effort, most often by predicting in one forward pass with the labeled rows as context. Existing surveys of tabular learning predate most of this fast-growing research or cover only part of it. We present a systematic survey: a PRISMA-guided pipeline screens 1,133 candidate records with two language models and human adjudication, yielding a corpus of 146 works. We arrange the corpus along two axes: the pretraining data source, synthetic priors or real tables, and the transfer mechanism, from in-context learning to fine-tuning, text generation, and weight generation. We describe 42 core models, group the 104 works that extend them into six threads, and review the benchmarks, including living leaderboards. Outside the corpus, we map the tasks these models reach beyond classification and regression. Synthetic-prior methods, popularized by TabPFN, dominate the field and have reached industrial scale, with company releases leading the live benchmarks. Open challenges include converting larger contexts into accuracy, combining numerical and semantic understanding, grounding prior and architecture design in theory, evaluating capabilities that no public benchmark covers, and keeping frontier models open to independent assessment as access to their weights and documentation narrows.

1. Introduction

Tabular data organizes observations along two axes: rows collect individual samples, and columns hold a fixed set of features that characterize each sample. The main strength of this representation is how little structure it imposes. Rows are typically assumed independent and identically distributed, columns carry no meaningful order, and the semantics of each column are set by the individual dataset rather than by a shared vocabulary. This freedom from geometric inductive bias, which sets tables apart from images, text, and video, is also what makes the modality ubiquitous in applied machine learning: electronic health records, financial transactions, customer databases, and inventory logs are all tabular, and tabular data has been ranked among the dominant modalities of the field, likely the largest in science [1].

Given this ubiquity, considerable research and industrial effort has gone into machine learning for tables, yet the reference solution has barely moved. For two decades, tabular prediction has belonged to the gradient-boosted decision tree, trained from scratch on each new table [2]. Deep neural networks, dominant in every other modality, have struggled to justify their training cost on tables [3]. In this context, foundation-model research has given tables little attention; a keyword estimate over three major machine learning conferences in 2023–2024 finds fewer than ten accepted papers per venue pairing foundation models with tables [1]. This neglect rested on a reasonable assumption: knowledge transfer, the mechanism on which every pretrained model is built, appeared ill-suited to tables. Text is built from a shared token vocabulary and images from pixels with shared low-level statistics; tabular feature spaces are instead heterogeneous, column order is arbitrary, and most tables are irrelevant to most other tables [1].

The introduction of prior-data fitted networks (PFNs) in 2021 marked a shift in this trend [4]. The core idea of a PFN is to pretrain a transformer on a distribution of synthetic prediction tasks until a single forward pass, with the labeled rows of an unseen dataset supplied as context, approximates Bayesian posterior prediction. This way, transfer does not require a representation shared across tables. The model learns a general prediction procedure during pretraining,

*Corresponding author
Email address: alberto.archetti@polimi.it (A. Archetti)
ORCID(s): 0000-0003-3826-4645 (A. Archetti); 0009-0005-7480-4102 (M. Mastroleo); 0009-0009-0482-6188 (M. Sabella); 0000-0001-6062-5174 (C. Cappiello); 0000-0002-8306-6739 (M. Matteucci)

and applies it to any new dataset it receives as input at inference time. TabPFN was the first widely adopted PFN, demonstrating that a pretrained model can be a valid alternative to gradient boosting [5]: a comparison of 19 algorithms across 176 datasets found it the strongest algorithm on average [6], and, at the time of writing, foundation models from six different organizations hold the first seven positions of the live TabArena leaderboard [7].

In this work, we present a systematic survey that tracks and categorizes the literature on PFNs and, more generally, tabular foundation models (TFMs). Following the PRISMA workflow [8], we query OpenAlex and Semantic Scholar for candidate works in the field. The scope of the survey covers methodological contributions to neural foundation models operating on tabular data exclusively: models pretrained once on a large corpus of synthetic or real problems that then adapt to unseen tables with minimal dataset-specific effort, regardless of feature and row count. We deliberately exclude generative models, models designed for tasks other than classification and regression, and direct applications of TFMs to new domains that make no methodological contribution to the paradigm itself.

After collecting all records with a cutoff of 15 September 2026, we let two language models screen each of the 1,133 candidate records independently against the survey scope. We adjudicate by hand every record that is not unanimously excluded, together with a sample of the unanimous exclusions, obtaining a final corpus of 146 curated papers. We organize the corpus along two orthogonal axes. The first is the pretraining data source, which separates synthetic priors, sampled from explicit generative programs over prediction tasks, from collected corpora of real tables, whether curated, crawled from the web, or derived from knowledge bases. The second is the transfer mechanism by which pretrained knowledge reaches a new dataset, which defines four model families: PFNs, fine-tuned encoders, language-model adaptations, and hypernetworks. Our categorization also tracks the dependencies between models, showing how the leading models derive from one another.

On top of the categorization of core models, we analyze the works that extend their capabilities, organized into six threads: designing the prior, what the models learn, reliability and robustness, interpretability, choosing the context, and adapting and scaling. Reading these threads against the limitations that the frontier models state about themselves lets us chart the open challenges of the current literature. Finally, reaching beyond the boundary of the PRISMA analysis, we map the problems that TFMs are beginning to serve past supervised prediction on a single table, including time series forecasting, data generation and imputation, survival analysis, causal inference, and relational data. Our ultimate goal is a reliable snapshot of the TFM literature that supports the open tracking of its best solutions.

Comparison with other surveys. A survey of deep tabular learning traces architectures from fully connected networks to attention-based designs, but its coverage ends in late 2024 and foundation models do not yet appear as a standalone category [9]. A broad TPAMI survey of tabular representation learning organizes the field into specialized, transferable, and general models, with a dedicated section on the general class that covers TabPFN and its variants; the latest works it covers date to early 2025, and its selection is expert-driven rather than systematic [10]. Several position papers frame the field, arguing that TFMs hold groundbreaking potential for tabular applications while remaining understudied, with real-world adoption still limited compared to well-established solutions [1, 11–13]. The closest work to ours is a critical review that reads the reported results of 14 current TFMs against independent evaluations of generalization, calibration, robustness to feature shift, and fairness [14]. Its taxonomy, by pretraining data and adaptation mechanism, is close to our two axes, and its conclusions agree with ours: current models are strongest on small and medium tables, and restrictive licenses on the strongest models limit independent evaluation. It is, however, a narrative review of works selected without a systematic protocol.

Our survey complements these works in three ways. First, it is systematic, replacing expert selection with a partially automated screening pipeline and a human-adjudicated corpus. Second, it covers TFMs exclusively, within an explicitly stated scope, covering both core models and extension works. Third, it categorizes the core models and tracks the dependencies between them, allowing future work to trace the line of contributions behind any architecture.

Outline and contributions. This survey makes four contributions, organized along its main sections.

1. Section 2 presents our semi-automated PRISMA pipeline, which screens 1,133 candidate records with two independent language models, adjudicates every disagreement and a sample of unanimous exclusions by hand, and yields a corpus of 146 curated works. Appendix D reports the full PRISMA checklist.
2. Section 3 introduces a taxonomy that locates the 42 core models along two orthogonal axes, the pretraining data source and the transfer mechanism.
3. Sections 4 and 5 organize the 104 extension works into six threads, together with a review of the benchmarks, including the living leaderboards that track the frontier.

4. Section 6 discusses the open challenges of the frontier models and maps the applications carrying the TFM paradigm beyond supervised classification and regression.

2. Systematic Review Methodology

The literature on tabular foundation models is expanding faster than manual curation can reliably track. We therefore adopt a semi-automated protocol that reserves human judgment for the steps where interpretation is required. The protocol follows the PRISMA workflow [8]: identification of candidate records, screening, and inclusion of the final corpus. In the screening stage, we combine automated screening by two language models from different vendors and human adjudication. Figure 1 summarizes the flow of records through the three stages.

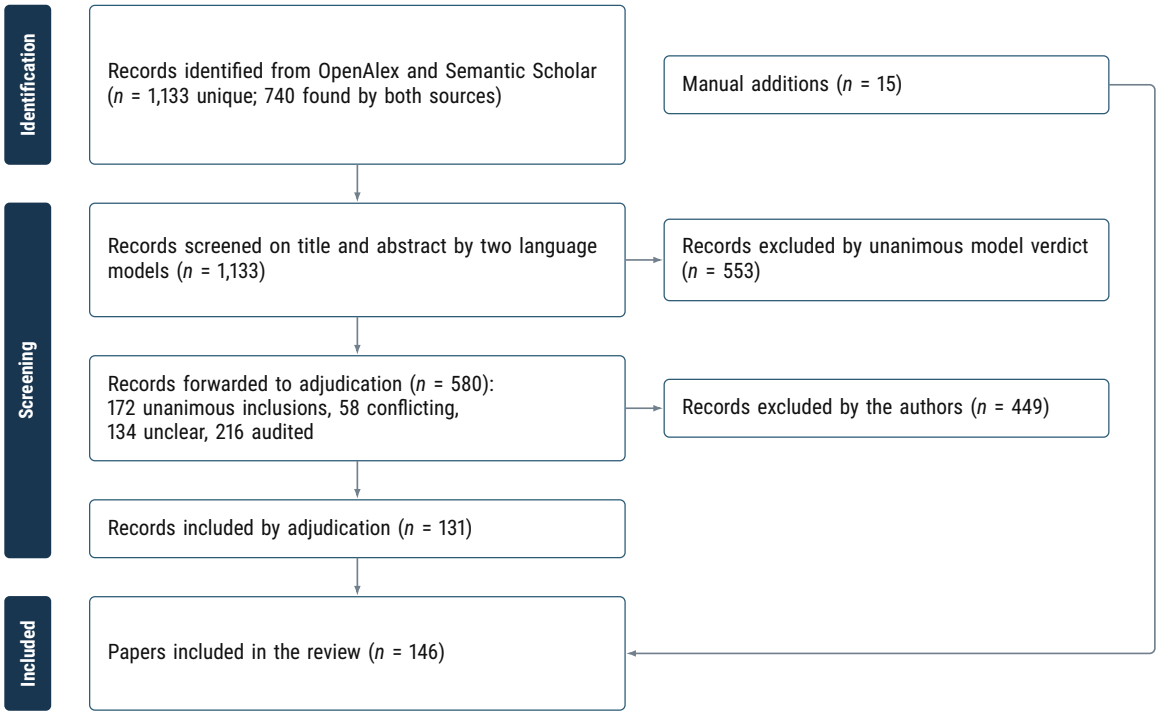


Figure 1: Flow of records through the systematic review, following the PRISMA workflow. Two bibliographic sources yield 1,133 unique candidate records, each screened by two language models. All included records have been human-adjudicated, resulting in a 146-paper corpus after a set of manual additions.

2.1. Identification

We queried two bibliographic sources, OpenAlex and Semantic Scholar, with "tabular foundation model", "TabPFN", and "prior-data fitted network" as the keyword set. This is the smallest set that recovers most of the relevant works in the field. The reason is that prior-data fitted networks were introduced by Müller et al. [4], while the first release of TabPFN [5] popularized them as a practical approach to tabular prediction. The consequence is a connected literature in which most subsequent contributions either build directly on TabPFN or adopt it as an experimental baseline. We ablated this choice by measuring the proportion of records recovered by a broader set of candidate terms, collecting our findings in Appendix A. The search was executed in September 2026 and records published after 15 September 2026 were discarded. After deduplication, the automated search returned 1,133 unique candidate records, 740 of which were found by both sources. Each record was enriched with citation counts and venue metadata to support the screening stages that follow. We complemented the automated search with 15 manual additions covering relevant works beyond the reach of the keyword set: industrial models documented only by a release page or a software deposit, such as Google's TabFM [15] and Nori [16], and studies whose titles and abstracts do not use the keywords [17–19].

2.2. Screening

Every candidate record was screened independently by two language models from different vendors, Claude Sonnet 5 (Anthropic) and GPT-6 Sol (OpenAI). We gave both models an explicit scope statement in which we defined what counts as a tabular foundation model and which tasks fall within the scope of the review. The exact prompt is reported in Appendix B. Working from title and abstract, with bibliographic metadata supplied as context, each model assigned two labels to the record: a category, chosen from eight predefined values (new model, model extension, application, benchmark, analysis, tooling, survey, out of scope), and a role played by tabular foundation models in the work (primary contribution, primary object of study, primary application method, baseline or mention, absent). Each assignment was accompanied by a one-line justification, and either label could be marked as unclear. The two labels determined a binary verdict. A record was included when it was not categorized as out of scope and a tabular foundation model was its primary contribution or primary object of study. Records that applied an existing model to a domain problem, mentioned one only as a baseline, or did not involve tabular foundation models at all were excluded.

Both models screened all 1,133 candidates. On the 999 records where neither model marked a label as unclear, the resulting verdicts agree in 94.3% of cases, with Cohen's $\kappa = 0.822$, indicating almost perfect agreement [20]. Claude Sonnet 5 was the more permissive screener, including 20.2% of all 1,133 records compared with 16.7% for GPT-6 Sol. Of the 769 unanimous exclusions, we audited 216 by hand, among them a random audit sample, and discarded the remaining 553; the audited records were forwarded to human review together with the 172 unanimous inclusions, the 58 conflicts, and the 134 records carrying an "unclear" label.

We manually reviewed the 580 records forwarded by the automated stage, examining titles and abstracts, and the full text whenever these were not conclusive. All 216 audited exclusions were confirmed. Of the 172 unanimous inclusions we retained 115, removing out-of-scope contributions and duplicated versions of the same work. Among the conflicting and unclear cases, we included 16 records. These steps resulted in a set of 131 papers retained from the automated screening. This set, together with 15 manual additions, forms the 146-paper corpus analyzed in the rest of this work.

2.3. The Resulting Corpus

The TFM literature is young and growing quickly. The corpus opens with a single paper in 2021, reaches 41 papers in 2025, and adds another 80 by mid-September 2026, projecting to 113 papers over the full year. Figure 2 splits this growth into core models and extension works. Core models, described in Section 3, introduce a new pretrained model, or a model version that improves substantially on its predecessors in method or in performance; works that only build pipelines around a released checkpoint count as extensions. Extension works build on core models, improving them or studying properties such as scaling, interpretability, or fairness. The split shows a maturing field: core models drive the early years, while extension works account for a growing majority from 2024 onward, up to 66 of the 80 papers of 2026. Section 4 organizes the extension works into thematic threads.

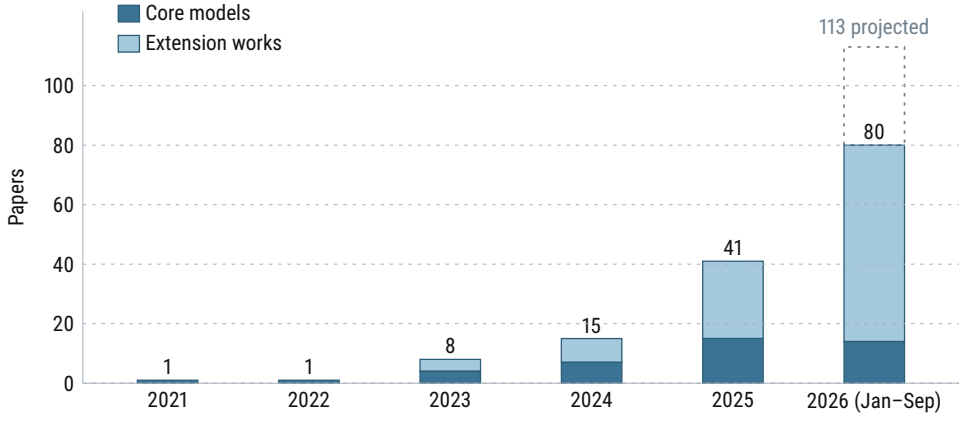


Figure 2: Papers in the corpus by publication year, split into core models, which introduce a new pretrained model or a substantially improved version of one, and extension works, which build on or study an existing core model. The 2026 bar covers January through 15 September; the dashed outline extends it to its linear full-year projection.

3. The Landscape of Tabular Foundation Models

This section describes the 42 core models of the corpus. Section 3.1 places them on the two axes of our taxonomy, tracing the dependencies that connect them. Section 3.2 examines their architectural backbones, and Section 3.3 the data they are pretrained on.

3.1. Core Models

We organize core models along two axes. The first is the pretraining data source. Twenty-six of the 42 core models pretrain on synthetic datasets sampled from an explicit prior over predictive tasks; the others collect real tables, from curated collections (10 models), web crawls (4), or knowledge bases (2). The second is the transfer mechanism, which defines four model families. PFNs form the largest family, with 28 of the 42 models. PFNs are based on in-context learning: labeled rows of the target dataset enter the pretrained transformer as context, and a single forward pass returns predictions for the query rows, with no weight update. Unlike PFNs, fine-tuned encoders, accounting for seven models, transfer pretrained representations and adapt to each table by gradient descent. Four models are adaptations that start from a pretrained language model and cast prediction as text generation over serialized rows. A final, smaller family of three models is the hypernetworks, whose pretrained network outputs the weights of a small, dataset-specific model that then makes the predictions. Appendix C lists the developer, size, supported inputs, and licenses of each core model.

Figure 3 arranges the core models on the resulting grid and draws an arrow from each model to those whose papers build on it. A single lineage originates from PFN and connects nearly all in-context learners and synthetically pretrained models, including MotherNet among the hypernetworks and MachineLearningLM among the language-model adaptations. The other models enter the graph as isolated nodes or small groups: the fine-tuned encoders, the other language-model adaptations, and the other hypernetworks, all pretrained on real tables, and three lines of synthetically pretrained in-context learners, namely LimiX, EXAONE Tabular, and Nori. The figure also shows how recent the field is: 29 of the 42 core models appeared in 2025 or 2026, and only PFN and TabPFN predate 2023.

3.2. Model Backbones

Nearly all core models are based on the transformer architecture. However, their backbones differ widely in structure. Three choices set them apart: what constitutes a token, along which axes attention operates, and how the pretrained network adapts to a new dataset, whether through a context of labeled rows, gradient-based fine-tuning, text generation, or the generation of a second network's weights.

Transformers. A transformer maps a set of input tokens to an equal number of output representations [21]. Each token enters as an embedding vector, and each layer updates it in two steps: self-attention, in which the token scores its compatibility with every other token and aggregates their representations with the resulting weights, and a position-wise feed-forward network. Two properties of this computation matter for tabular models. First, self-attention treats its input as an unordered set, so without positional encodings the model is equivariant to token permutations. This is a natural fit for tables, where rows and columns carry no meaningful order. Second, the standard formulation of attention compares every pair of tokens, so its cost grows quadratically with their number, and the choice of what a token represents, a row, a cell, or an intermediate aggregate, determines how the model scales with the size of the table. The architecture comes in two main variants: encoders, which process the whole input at once and output one representation per token, and decoders, which restrict attention to preceding tokens and generate output autoregressively.

Prior-data fitted networks. A substantial part of the corpus is based on PFNs, building on a Bayesian view of prediction [4]. Here, every dataset is assumed to be produced by some underlying mechanism, treated as a latent task t. Since the true mechanism is unknown, one specifies a prior $p(t)$, a distribution stating in advance which mechanisms are plausible. Given a labeled dataset $D = \{(x_i, y_i)\}_{i=1}^{n}$, the Bayesian prediction for a query x is the posterior predictive distribution (PPD),

$$p(y \mid x, D) = \int_t p(y \mid x, t)\, p(t \mid D)\, \mathrm{d}t, \tag{1}$$

which averages the predictions of all possible mechanisms, each weighted by $p(t \mid D)$, its probability after observing the data. This form of prediction has two classical benefits: calibrated uncertainty and assumptions made explicit in the prior. However, except for the simplest priors, the integral cannot be computed explicitly. Standard methods approximate the

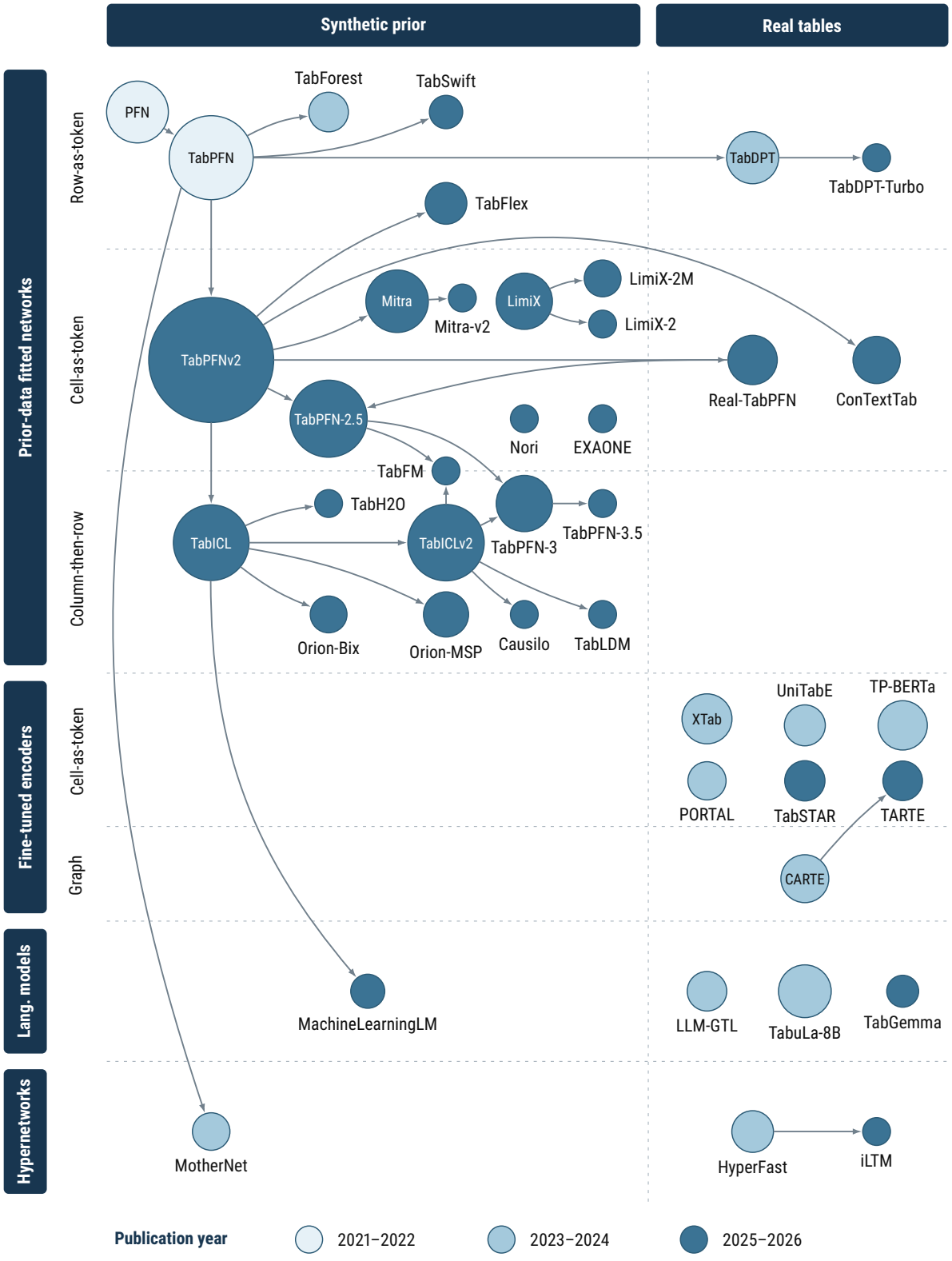


Figure 3: The 42 core models of the corpus arranged along the two axes of the taxonomy: the pretraining data source (columns) and the transfer mechanism, whose values are the four model families (rows). The area of each node is proportional to the citations per year of the introducing paper (Semantic Scholar, September 2026), the fill encodes the period of first release, and an arrow points from a model to one whose paper builds on it.

posterior $p(t \mid D)$ instead, by sampling it with Markov chain Monte Carlo (MCMC) or by fitting a tractable surrogate with variational inference, a substantial computational cost paid again for every new dataset [4].

A PFN follows a different approach: it never approximates the posterior and instead trains a neural network to output the PPD directly. The key observation is that the prior supplies unlimited training material. One can sample a mechanism, generate a labeled dataset from it, hide one label, and ask the network to predict it. A transformer q_θ receives the dataset D and the query x as a single input and is trained, across millions of such sampled datasets, to minimize the prior-data negative log-likelihood

$$\ell_\theta = \mathbb{E}_{D \cup \{(x,y)\} \sim p(D)} \big[-\log q_\theta(y \mid x, D) \big]. \tag{2}$$

It can be shown that minimizing this loss drives q_θ toward the exact PPD of Equation 1 [4]. After pretraining, a single forward pass, with the labeled dataset supplied as context, returns an approximate Bayesian prediction. Two consequences follow. First, all learning moves to pretraining: fitting a new dataset requires no weight update, so its cost is one forward pass instead of an optimization loop. Second, the only requirement on the prior is the ability to sample datasets from it, so any program that generates datasets defines a usable prior, however intractable its posterior. On priors where the exact answer is known, such as Gaussian processes and Bayesian neural networks, PFNs approximate the PPD orders of magnitude faster than MCMC or variational inference [4].

Architecturally, a PFN is a transformer encoder without positional encodings: every labeled context row and every query row is a token, and an attention mask lets context rows attend to one another while query rows attend only to the context [4, 5]. The output position of each query feeds a prediction head, a softmax over classes or a piece-wise constant distribution over the regression range [4].

Row-as-token PFNs. The original PFN layout flattens each row into one token. PFN and TabPFN project the feature vector, zero-padded to a fixed width, through a linear layer, and context rows add an embedding of their label [4, 5]; TabPFN instantiates the design as a 12-layer encoder with 26 million parameters, targeting datasets of up to 1,000 rows, 100 features, and ten classes [5]. The fixed projection width is what caps the feature count in row-as-token transformers. TabForest isolates the effect of pretraining data while keeping the TabPFN architecture [22]. TabDPT improves PFN context scaling by supplying only the nearest training rows as context for each query [23]. TabDPT-Turbo drops the retrieval and scales the same backbone to contexts of 100,000 rows [24]. TabFlex swaps softmax attention for linear attention, removing the quadratic cost and reaching datasets with over a million rows [25]. TabSwift adds learned register tokens and gated attention and shows that row-wise attention alone stays close to the column-aware models below at a lower inference cost [26].

Cell-as-token PFNs. TabPFNv2 refines the token to a single cell and alternates attention along the two axes of the table: each cell attends first to the other cells of its row and then to its feature column across samples [2]. Cell tokens remove the fixed feature width, let the model handle heterogeneous types and missing values cell by cell, and raise the supported size to 10,000 rows and 500 features in the second TabPFN iteration [2]. Real-TabPFN builds on TabPFNv2 by including in its pretraining data a curated set of real tables [27]. TabPFN-2.5 deepens the network to up to 24 layers and prepends 64 learned "thinking" rows, giving the model additional computational capacity and reaching 50,000 rows and 2,000 features [28]. ConTextTab keeps the alternating layout but embeds each cell with a pretrained sentence encoder and adds an embedding of the column header in place of positional encodings, making the backbone semantics-aware [29].

Mitra focuses on redesigning the prior entirely [30]. Its successor, Mitra-v2, keeps the 12-layer backbone of Mitra, at about 77 million parameters, and scales the pretraining tasks further; by default, the checkpoint is fine-tuned and bagged on each dataset rather than used in a single forward pass [31]. LimiX applies two column attention passes for every row attention pass [32], and LimiX-2M introduces radial-basis cell embedding, avoiding the representation collapse found in standard scalar cell embedding [33]. LimiX-2 scales the LimiX backbone to 24 blocks with 256-dimensional cells, up to 406 million parameters, and separates the pathways of features and target: the target is split into four embedding slots that read from the feature cells, and a low-rank code per column replaces positional information [34]. Nori groups features in pairs, embeds values with radial-basis functions, and specializes the architecture to probabilistic regression through a 999-quantile output head, leading to a substantially smaller model than TabPFNv2 at 6 million parameters [16]. EXAONE Tabular keeps one token per cell throughout its 12 layers: each layer applies two attention passes within each row and one along each feature column, in which query cells read only from the context cells, and three summary

tokens per row and 32 per feature carry information between the two axes. Classification and regression use separately trained checkpoints of about 21 million parameters each, and the classifier predicts up to ten classes [35].

Column-then-row transformers. While highly accurate, cell-as-token PFNs require a token count proportional to rows and columns. TabICL avoids this by staging the computation in three encoders [36]. A column-wise set encoder summarizes the distribution of each feature; a row-wise encoder then compresses each row into a fixed-size embedding through learned aggregation tokens; only then does an in-context transformer attend across rows, so the dataset-wide attention operates on one token per row. The design matches TabPFNv2 in accuracy while running up to 10× faster and scaling to 500,000 rows [36]. TabICLv2 refines every stage: it injects the training labels into the column encoder already at the first stage, rescales attention to counteract its fading in long contexts, predicts regression targets through 999 quantiles, and handles million-row tables [37]. TabPFN-3 abandons the alternating layout of its predecessors for this staged design, which it adapts from TabICLv2, and adds an attention-based decoder for up to 160 classes and a chunked inference path that reaches one million rows in context [38]. TabPFN-3.5 keeps this staged design and revises the cell encoding and the width. Each value is encoded with learned Fourier features and with its empirical rank among the training values of its column. This encoding makes the representation invariant to monotone transformations and removes the quantile, scaling, and SVD preprocessing of earlier versions. The in-context transformer doubles in width, for a model of 220 million parameters [39].

The remaining members rework single stages of the TabICL architecture. Orion-Bix focuses on richer feature interactions via full, grouped, and cross-partition attention, whereas Orion-MSP focuses on scalability to many features through multi-granularity block-sparse attention with shared memory across stages [40, 41]. TabH2O merges classification and regression into a single dual-head model trained in a single stage on data that includes explicit noise columns [42]. TabFM applies an alternating-attention layer on cell tokens, then compresses the rows and performs in-context learning over the compressed rows [15]. We therefore represent it as a hybrid between cell-as-token and column-then-row PFNs. Two further models modify every stage of TabICLv2. Xiaomi-TabLDM adds a second feature-grouping stream, learned attention residuals, and a sparse mixture of experts in the last eight of its 24 in-context blocks, for about 70 million total and 62 million active parameters [43]. Causilo refines the cell tokens before compressing them: four summary tokens per row exchange information among the cells of the row, and a second column stage lets the refined cells revisit the context. Within-row attention thus grows linearly with the number of features, in a model of 36 million parameters [44].

Fine-tuned encoders. The encoder family adapts BERT-style architectures established for text to tabular data. These models pretrain a transformer encoder on a large corpus, then fine-tune it on downstream tasks [45]. Each cell, encoded together with its column name, becomes a token. Knowledge transfers through the pretrained cell representations, and adaptation requires fine-tuning, which separates this family from PFNs. XTab shares a three-layer backbone across datasets and re-initializes a feature embedding layer and a head per table [46]. UniTabE encodes each cell as a key-value pair of column name and value and scales the encoder to billions of Kaggle examples [47]. PORTAL encodes each cell with one type-specific embedding, a pretrained sentence embedding for text, calendar features for dates, or a sign-fraction-exponent encoding for numbers, summed with an embedding of the column header, which removes the need for feature normalization and for special handling of missing values and outliers [48]. TP-BERTa adapts the RoBERTa architecture and weights to tabular settings: numbers are discretized into magnitude tokens added to the vocabulary and scaled by their value, and an intra-feature attention module fuses each feature name and value into one token before the encoder [49]. TabSTAR unfreezes the upper layers of a pretrained text encoder, verbalizes features and candidate targets, and shares all parameters across tasks, so that only lightweight adapters are tuned per table [50]. CARTE constrains attention with a graph: each row becomes a small star graph whose nodes are the cell values and whose edges carry the column names [51]. Unlike models pretrained on collections of tables, CARTE is knowledge-pretrained on YAGO [52], turning millions of real-world facts represented as head-relation-tail triples into graphlets and learning their representations contrastively. Its successor TARTE drops the graph, letting every cell of the row attend to every other and extending the same knowledge-pretraining idea to a broader corpus [53].

Language-model decoders. The language-model adaptations use a decoder pretrained on text as the backbone: rows are serialized into strings, they enter the prompt as in-context examples, and prediction is the generation of the label tokens. This way, the causal attention of the decoder takes the place of the PFN masking scheme. The approach traces back to TabLLM, which serializes rows and fine-tunes a language model separately on each dataset [54]; because it adds

no tabular pretraining stage, it does not count as a core model in our corpus. LLM-GTL instead continues pretraining Llama 2 on instruction-style serializations of hundreds of tables, applying the next-token loss to feature-value and target tokens while using task and feature descriptions only as conditioning context [55]. Unlike PFNs, which require labeled context examples at inference, these language-model approaches can also operate in a strict zero-shot setting with no labeled rows, relying on the task and feature semantics encoded in language. TabuLa-8B extends Llama 3-8B with a row-causal attention mask that lets each serialized row attend to the previous rows of the same table, training few-shot behavior directly [56]. TabGemma follows a similar approach to TabDPT, pairing Gemma 3 with context retrieval that selects 128 similar rows for each query [57]. Whereas these models pretrain on real tables, MachineLearningLM targets many-shot in-context learning by continuing Qwen2.5 on millions of synthetic causal tasks, warm-started from a random-forest teacher, and packs up to 50 queries behind a shared set of demonstrations in each prompt [58].

Numerical values in TabLLM, LLM-GTL, and TabuLa-8B are serialized directly as text and handled by the backbone tokenizer, although TabuLa-8B discretizes continuous prediction targets into quantile bins [54–56]. The more recent models explicitly regularize numerical inputs: TabGemma renders every number in signed scientific notation with four significant digits [57], whereas MachineLearningLM standardizes each numerical feature and discretizes the result into an integer in $[0, 999]$, reducing the number of tokens required per value [58].

Hypernetworks. Hypernetwork models do not output predictions directly: given a labeled dataset, the pretrained network generates the weights of a smaller network, usually a multilayer perceptron, that is then evaluated on the query rows. MotherNet keeps a TabPFN-style row encoder, averages the row embeddings per class into a dataset embedding, and decodes it into the low-rank weight factors of a two-hidden-layer perceptron [59]. HyperFast and iLTM are the only core models that do not build on a transformer-based backbone [60, 61]. HyperFast projects each row through random features and a PCA fitted on the context rows, obtaining a fixed-width, feature-permutation-invariant representation, and then generates a residual perceptron one layer at a time [60]. iLTM keeps this design and adds a gradient-boosted-tree embedding, which encodes each row by the leaves it reaches across the ensemble, and a retrieval term that interpolates the generated network logits with a soft nearest-neighbor score over the context rows [61].

3.3. Training Data

What a tabular foundation model knows is defined during pretraining, by the data it is trained on before it sees any target dataset. The pretraining data source, the first axis of the taxonomy, separates synthetic priors, sampled from a generative program over prediction tasks, from collected corpora of real tables. A second choice, orthogonal to both axes, is the prediction target. A target may be designated in advance, by construction in a synthetic prior or by the known target column of a real table, or it may be drawn from the table itself, by hiding a random column or random cells and predicting them, which turns pretraining into a self-supervised objective. This section follows the columns of Figure 3: the synthetic priors first, then the collected corpora, curated, crawled from the web, or derived from knowledge bases, and finally the choice of target that collected tables leave open.

Synthetic priors. The only requirement of the PFN loss (Equation 2) is the ability to sample datasets. Thus, any generative program over datasets is a valid prior. The PFN lineage pioneered the systematic exploration of tabular synthetic priors. The original PFN proposes Gaussian processes and Bayesian neural networks as analytically motivated priors [4]. TabPFN introduces structural causal models (SCMs): a random directed acyclic graph is sampled, random mechanisms propagate noise through it, and features and target are read off its nodes. In TabPFN, SCMs are mixed in equal proportion with Bayesian neural networks, and simpler graphs are made more likely [5]. SCMs have since become the standard choice, appearing in almost every synthetically pretrained core model. Tree-based generators have been introduced to complement SCMs: TabForest overfits decision trees to random noise, producing deliberately unrealistic datasets whose axis-aligned boundaries improve fine-tuned performance [22]. TabICL inserts gradient-boosted regressors among the SCM mechanisms and samples 30% of its datasets from these tree-based SCMs [36]. TabPFNv2 diversifies SCM mechanisms with small neural networks, discretization steps, and decision trees [2]. EXAONE Tabular is pretrained entirely on synthetic tables generated from causal models, with transformations that produce diverse feature types, noise, and missing values [35].

Because the prior largely determines what the model learns, its design has become a research direction of its own. Mitra selects its mixture by measuring, for each candidate generator, the downstream performance of a model pretrained on it alone, the diversity of its samples, and its distinctiveness from the other generators, and proposes an equal mix of causal and tree-based priors [30]. Mitra-v2 pairs the TabPFN generator with a hybrid causal model

adapted from an extension work (Section 4.1) [62], in which each node of the graph applies its own mechanism, from neural networks and tree ensembles to Gaussian processes and vector autoregression [31]. TabICLv2 rebuilds the generator around eight function families and filters out datasets that a small forest cannot predict better than a constant baseline; its ablations attribute the largest single effect to the prior, but prior and architecture are coupled: the new architecture pretrained on the earlier TabICL prior falls below TabICL, and the TabICL architecture pretrained on the new prior only matches it [37]. TabPFN-3 extends the prior toward hundreds of classes, temporal structure, and out-of-distribution extrapolation [38], and TabPFN-3.5 toward high-cardinality categorical features, wide tables, and grouped data whose test rows come from groups unseen in training [39]. LimiX generates data from hierarchical SCMs whose mechanisms include convolutions and decision trees [32], and LimiX-2 composes its graphs from recursively expanded causal motifs, such as chains, confounders, and colliders, adding linear, kernel, piecewise, periodic, and multiplicative mechanisms [34]. MachineLearningLM brings synthetic data into the language-model family, continuing pretraining on three million SCM-sampled tasks [58]. The corpora are large: TabPFN pretrains on 9.2 million sampled datasets [5], TabPFNv2 on 130 million [2], TabICL on 83 million [36], Mitra on 45 million [30], TabICLv2 on 35 million [37], Causilo on 36 million [44], Mitra-v2 on about 29 million [31], EXAONE Tabular on about 30 million for classification and 10 million for regression [35], and TabFM, according to its release note, on hundreds of millions [15].

Curated tables. At the other end of the data axis, the models pretrained on real tables pair each with a curated corpus. XTab pretrains its shared backbone on 52 tables of the OpenML-AutoML Benchmark [46], UniTabE collects 283,000 Kaggle tables totaling 13 billion examples [47], TP-BERTa draws 202 datasets with about ten million samples from a semantic table database [49], and TabSTAR curates 350 datasets chosen to include tables with textual fields [50]. TabDPT brings curated data to the PFN family, pretraining on 123 OpenML datasets with 32 million rows and reporting power-law scaling in both model and data size [23]; TabDPT-Turbo grows the corpus to 1,445 datasets and 310 million rows [24]. The hypernetworks meta-train on labeled collections: HyperFast on OpenML-CC18 tables and genomic datasets [60], and iLTM on 1,806 OpenML datasets retained from roughly 5,000 after deduplication [61]. LLM-GTL serializes 340 Kaggle datasets for its continued language-model pretraining [55]. Curated tables also refine synthetic checkpoints. Real-TabPFN continues the pretraining of TabPFNv2 on 71 datasets curated from OpenML and Kaggle; in its ablation, a corpus of 97,000 small CommonCrawl tables hurts performance where the curated corpus helps [27]. TabPFN-2.5 applies the same procedure: its companion model, Real-TabPFN-2.5, continues pretraining on 43 curated datasets [28]. TabPFN-3 and TabPFN-3.5, by contrast, do not report using any real tables in pretraining [38, 39].

Web corpora and knowledge bases. Unlike curated tables, web corpora trade careful selection for scale. TabLib is a collection of 627 million tables from CommonCrawl and GitHub [63]; TabuLa-8B filters out about 98% of them, keeping over four million tables with 2.1 billion rows as the T4 corpus [56]. Both ConTextTab and TabGemma adopt T4: the former retains only the 2.18 million tables with at least 150 rows [29], whereas the latter pretrains on the full corpus [57]. PORTAL, by contrast, crawls 8.6 million tables from Wikipedia infoboxes and Wikidata [48]. The knowledge-base pair replaces tables with entity graphs: CARTE pretrains contrastively on YAGO3, with 18.1 million triples over 6.3 million entities [51, 52], and TARTE moves to YAGO4.5, which it enriches with numerical facts from Wikidata, reaching 30 million facts over 5.5 million entities [53, 64].

Selecting the pretraining targets. Core models obtain their pretraining targets in one of two main ways: supervised training on a predefined target column, or self-supervised training on targets derived from the table itself, whether a randomly or heuristically chosen column or a set of masked cells. Synthetic priors provide the target by construction, since every sampled dataset designates a target node. Collected tables do not always define an explicit target. In this case, the most direct choice is to pick a random column as the prediction target, as TabDPT [23], TabDPT-Turbo [24], ConTextTab [29], and TabGemma [57] do. Instead of random selection, TabuLa-8B fixes one target column per table through heuristics [56]. XTab, UniTabE, and PORTAL reconstruct masked or corrupted cells [46–48], and LLM-GTL models feature values and targets jointly as one autoregressive sequence [55]. TP-BERTa and TabSTAR are pretrained with supervision, exclusively on tables whose target column is known [49, 50], as are the meta-trained hypernetworks HyperFast and iLTM [60, 61]. Masking is not tied to collected data, either: LimiX masks random cells of its synthetic tables and predicts them jointly, so a single pretrained model serves prediction, imputation, and generation [32]. LimiX-2 keeps this objective and interleaves three masking schemes, over single cells, whole columns, and blocks [34]. The contrastive models CARTE and TARTE follow a third strategy: each entity graph is matched against a truncated copy of itself and pushed apart from the other graphs in the batch, which removes the need for a target entirely [51, 53].

4. Extension Works

Beyond the 42 core models, the remaining 104 papers of the corpus build on them, improving the models or studying their properties. This section synthesizes their findings. We organize them into six threads: designing the prior, what the models learn, reliability and robustness, interpretability, choosing the context, and adapting and scaling. The benchmarks among the extension works are reviewed separately in Section 5.

4.1. Designing the Prior

Building on Section 3.3, a group of extension works isolates the design of the prior as the experimental variable. O'Prior holds the architecture, optimizer, and compute budget of nanoTabPFN [65], a compact reimplementation of TabPFNv2, fixed and varies only the generator: the diversity of structural mechanisms accounts for most of the gain, whereas observational realism, such as heavy tails, missingness, and label noise, and explicit train-test shift add smaller, independent gains. A curriculum from simple to hard tasks is not uniformly better than a static prior under a fixed budget [62]. A variant of the hybrid causal generator introduced by O'Prior has since been adopted by Mitra-v2 [31]. The prior can also be widened adversarially, either during or after pretraining. During pretraining, part of the random generators can be replaced with agents updated by gradient ascent on the model loss, which spreads the data distribution toward datasets the current model handles poorly [66]. After pretraining, the generator configurations on which the model loses to conventional baselines fitted on the same synthetic data can be identified and used for continued pretraining with a small fraction of the original budget [67].

An intuitive measure of prior quality is how closely its samples resemble real tables. A study of the TabICL prior, the only one with open parameters, finds that a discriminator separates synthetic from real tables almost perfectly and that tuning the prior hyperparameters does not eliminate the difference; yet datasets far from the prior are predicted as well as datasets close to it, whether the distance is measured with hand-crafted features or with the embeddings of TabICL itself. This suggests that a prior transfers inductive biases about functional structure rather than coverage of real-table statistics [68]. The same study finds that curated and web-scraped corpora cover the same regions of table space, which explains why a few hundred curated tables match web-scale pretraining. A controlled comparison that pretrains the same nanoTabPFN on eleven priors, from those of TabPFN, TabICL, and TabForest to real OpenML tables, points the same way: the synthetic priors perform almost identically, and real tables pretrain better with randomly chosen targets than with their default ones [69]. Across priors, however, closeness to the benchmark is associated with relative performance. When synthetic and benchmark datasets are described by the same structural statistics, the TabICLv2 prior lies closest to the largest share of the datasets of TabArena and TALENT [70], and the model whose prior lies closer to a dataset usually ranks better on it [71]. Similarity to a known target domain can also be exploited: in the embedding space of TabPFN-2.5, synthetic tasks are almost perfectly separable from engineering regression datasets, and continuing pretraining on the few generated tasks closest to them, with no real rows, lowers the error on most of these datasets [72].

The prior also encodes assumptions about deployment, and several works adjust either the prior or the data when these assumptions fail. Drift-Resilient TabPFN samples a second causal model that shifts the parameters of the first over time, so that a single forward pass extrapolates temporal drift [73]. StablePFN exploits the causal graph that the synthetic prior already samples. An auxiliary head learns to identify the Markov boundary of the target, the smallest set of features given which the target is independent of all other features, and at inference the prediction branch attends only to those features. When a spurious correlation between the target and a feature outside the boundary reverses at test time, StablePFN still predicts well, whereas TabPFN-2.5 and LimiX fail [74]. When the checkpoint cannot be retrained, the data can be adapted to the prior instead. In strategic settings, where individuals modify their features to obtain favorable predictions, pairing each context row with a simulated manipulated copy of itself aligns the context with the deployment distribution [75]. For fine-tuning on a small table, augmenting the batches with samples from a causal model fitted to the table makes validation performance a reliable early-stopping signal again [76]. For dirty data, a learned policy performs cleaning operations to move the table closer to the prior [77].

4.2. What the Models Learn

Models with near-identical benchmark scores can implement different prediction rules. A mechanistic study of TabPFNv2, TabICLv2, and Mitra finds that each model reads its prediction off its representation differently: TabPFNv2 and Mitra take an attention-weighted vote over the context labels at a single late layer, whereas TabICLv2 assigns the query to the nearest class prototype [78]. The readouts are not interchangeable, and each predicts its own failure: replacing feature values with their ranks preserves order but destroys scale, and it hurts TabPFNv2 and TabICLv2

while leaving Mitra unaffected, since Mitra quantizes its inputs. The same mechanistic study shows that the missing symmetries of the released checkpoints cost nothing to restore: zeroing one positional parameter makes TabPFNv2 and TabICLv2 exactly invariant to column order, and a one-versus-all wrapper makes any of the three invariant to class order, at unchanged accuracy and without the permutation ensembling that current pipelines pay for [78]. EquiTabPFN goes further and builds class-order equivariance into the architecture, which lets the model handle more classes than it saw in pretraining [79].

Depth is largely redundant. Skipping the first layers damages the model beyond recovery, whereas skipping middle or late layers costs little, because later layers repair the damage and overlap in what they compute [80, 81]; at small scale, a single transformer block applied repeatedly matches a six-block model [81]. This redundancy is what makes the early-exit, pruning, and distillation methods of Section 4.6 viable. Inside the layers, only a small fraction of neurons respond selectively to a single concept, and these neurons are spread across all depths [82, 83], while intermediate terms of the target function are linearly decodable from the middle layers [84]. TabPFNv2 can also serve as a feature encoder: a linear classifier on its embeddings matches the in-context predictions [85]. These embeddings depend on the labels, however: row embeddings extracted from TabPFNv2, TabICL, and TabSTAR with a random dummy target perform no better than a simple one-hot encoding when fed to nearest-neighbor or MLP heads [86].

Finally, the inductive biases of the models can be read off their outputs. Plotting the functions TabPFN fits to simple datasets reveals predicted probabilities that wiggle between context points and a bias against periodic structure [87]. In the frequency domain, the range of functions TabPFN can represent grows with the number of context samples [88], and the frequency of a regression input is linearly decodable from the attention scores of TabPFN-2.5 and carried by a few principal directions that causally determine the prediction [89]. On data generated from physical equations, TabPFN-3, TabICLv2, TabDPT, and Real-TabPFN-2.5 outperform tuned baselines within the sampled range, but they do not generalize beyond it. On noiseless data, their uncertainty also does not vanish, since their predictive intervals stop shrinking even as the error keeps falling. The authors attribute this to the prior, which adds noise to every mechanism and therefore cannot represent a deterministic law [90]. The size of the context shapes what the model can learn: context rows beyond the pretraining window keep reducing the variance of TabPFN predictions but not their bias, which explains the accuracy plateau of the first TabPFN on large datasets and why restricting the context to the neighbors of each query resumes the improvement [91]. Models pretrained on a single real table suggest a different mechanism: the better a model retrieves the label of a context row presented again as a query, the higher its downstream accuracy. The authors therefore describe TFMs as learned procedures that retrieve and combine similar rows, not as approximate Bayesian inference [92]. Attention maps must be read with caution. When features are correlated, attention distributes among them while accuracy stays the same, so attention does not measure feature importance [93]. Conversely, sharp attention does not make a component necessary: on synthetic regression tasks, a head of TabPFN-2.5 that attends to few features can be removed with almost no effect on the prediction [94].

4.3. Reliability and Robustness

A separate line of work asks how far the predictive distributions of TFMs can be trusted, and finds that high accuracy does not guarantee reliable uncertainty. Stacking the current models buys some accuracy but destroys calibration, and ensembling them gains little because they agree on most inputs [95]. TFMs also reach the highest AUC but lower conformal coverage than gradient-boosted trees [96]. Their predictions are not even internally consistent: when one target is used to predict another, the implied joint distribution depends on the order of prediction, largely regardless of accuracy [97]. Most benchmarks do not measure these properties, since they still score regression with point metrics instead of proper scoring rules [98], with ScoringBench as the exception for distributional regression (Section 5). Against Gaussian processes, TabPFN is competitive on complex, higher-dimensional problems with more data, whereas the Gaussian process wins when data are scarce or its kernel fits the function [99]. Beyond the label, martingale posteriors quantify uncertainty about summaries such as conditional quantiles [100], and a predictive central limit theorem separates epistemic from aleatoric uncertainty. The theorem tracks how the predictions of TabPFN-2.5 change as context rows are added one at a time, although its conditions are verified only on a small-scale toy model trained on a coin-flip prior [101].

A second question is how predictions hold up under distribution shift. TabPFN tolerates synthetic stress such as uninformative features, correlated feature groups, and label noise [93]. Real shift, in contrast, degrades all nine models evaluated on the TableShift benchmark, and pretraining on real data raises accuracy without narrowing the gap between in-distribution and out-of-distribution performance [102]. TabPFNv2 also degrades faster than tree and neural baselines when features are missing at test time or under concept shift [103]. TFMs are the strongest option on small and medium

independent and identically distributed tables, but tuned tree and neural baselines still win on temporal, grouped, large, high-dimensional, and high-cardinality tasks [104], and dataset meta-features barely predict which family wins where [105].

Several reliability gaps can be mitigated, often at negligible cost, when the deployment condition is known. Under label shift, the model overfits to the class distribution of its context, and rescaling the predicted probabilities at test time to downweight this prior restores accuracy without hurting the unshifted case [106]. Under class imbalance, loss-based corrections are unavailable to an in-context learner, but moving the decision threshold works well because the predictions are calibrated, and downsampling the majority class works comparably [107]. For fairness, selecting context examples whose sensitive attribute is hard to predict improves group fairness at a small cost in accuracy, whereas decorrelating the features from that attribute can make fairness worse [108]. Penalizing the prediction of a sensitive attribute during pretraining also reduces unfairness, but lowers accuracy to that of logistic regression [109]. For privacy, a fine-tuned model memorizes its fine-tuning data only under artificial training procedures, and memorization vanishes under realistic ones [110].

4.4. Interpretability

Some models are interpretable by construction, and the constraint costs little accuracy. GAMformer outputs the shape functions of a generalized additive model in a single forward pass and matches conventionally fitted additive models [111]. A TFM can also select the feature interactions of an explainable boosting machine, which works better than the standard greedy selection [112]. Replacing the prediction head of a TFM with an explicit kernel makes every prediction a weighted average of context labels, with no measurable loss in accuracy [113]. Post-hoc explanations, in turn, become cheaper for an in-context learner, since refitting on a subset of features takes a single forward pass. Kernel SHAP can then retrain exactly instead of imputing the absent features, which lowers both its error and its runtime [114], and building Shapley value regression into the architecture returns predictions and attributions together, orders of magnitude faster than Kernel SHAP [115]. When the model to explain is inaccessible, a PFN pretrained on datasets labeled with exact Shapley values predicts the attributions a typical model would assign from the data alone [116].

4.5. Choosing the Context

For an in-context learner, the labeled context is the entire interface between the pretrained model and the downstream task, and a line of work asks how to select it when the training data exceeds the size the model supports. Simple methods suffice: random subsampling performs about as well as clustering-based selection, coresets, and farthest-point selection [117, 118]. What matters is how well the context covers the feature space, not how closely it matches the training distribution [118], and reducing the feature count helps more than any row-selection scheme [117].

Beyond subsampling, three families of methods have emerged. The first learns a global context: TuneTables distills the dataset into a soft prompt for a frozen TabPFN, which also lifts its limits on samples, features, and classes [119]; the context can also be optimized in input space by gradient descent through the frozen model [120], or selected without gradients by estimating Shapley values for rows and features, which, with the first TabPFN, outperforms both simple selection and the full training set [121]. The second retrieves a local context for each query. Supplying each test point with its nearest labeled neighbors improves accuracy even when the full dataset fits in context, because a small local context compensates for the limited expressivity of a single forward pass [122]. Local contexts also reduce cost: TabPFN and TabDPT match their full-context accuracy with a few dozen rows per query when the number of rows adapts to how mixed the labels are around the query [123], and TabICLv2 retains nearly all of its accuracy when it retrieves neighbors in its own row-representation space [18]. The third recovers batched inference: clustering the test queries and matching a distributionally aligned training subset to each cluster approaches per-query retrieval at a fraction of the forward passes and is resilient to covariate drift [124].

The memory limit on the context is partly a matter of implementation. Computing attention exactly in chunks, as fused attention kernels do on recent GPUs, lets the released TabPFNv2 weights process a hundred thousand rows, and accuracy continues to improve beyond the pretraining context size on about half of the datasets tested and levels off on most of the others [125]. Chunking, however, lowers only the peak memory: the number of operations still grows quadratically with the context, so on large tables the cost of inference, not whether it fits in memory, becomes the binding constraint. Methods that reduce the context should therefore be compared with the full context in accuracy, and judged by what they save in compute. Compression pays off in particular when the same training set is queried many times: TACO compresses it once into a small latent representation, after which predictions are far faster and lighter in memory at no loss in accuracy [126].

The context can also draw on data beyond the labeled training set. When only a few rows can be annotated, choosing them in the embedding space of the model gives a better context than choosing them in the raw feature space [127]. Unlabeled rows can enter with the model's own soft pseudo-labels, which helps when labels are scarce [128], and a small target table can borrow rows from a larger, shifted source table, provided they are selected by optimal transport with a penalty on label mismatch, since random source rows make the predictions worse than the target rows alone [129].

4.6. Adapting and Scaling

When a frozen checkpoint underperforms on a new dataset, the cheapest option is to reshape only the input. A small residual adapter shifts the input toward the distribution the model expects [130], an ensemble of lightweight encoders removes the limit on the number of features [131], and a second frozen TabPFNv2 with a trainable encoder can learn the residuals of the first, which also combines data of different fidelity [132]. Hand-crafted feature engineering, by contrast, helps less with every model release [133]. At the other end of the cost scale there is fine-tuning. On TabPFNv2, tuning all weights beats parameter-efficient alternatives because it improves the retrieval of relevant context rows [134], and a gradient step on the context itself lets the model reach nearly the same accuracy with a much smaller context, a gain that theory ties to the alignment between the prior and the target task [135]. The best strategy is model-specific, however. Supervised fine-tuning helps TabPFN but makes TabICL overfit, whereas parameter-efficient tuning retains most of the benefit at a fraction of the cost [136]. On small general-domain tables, fine-tuning even hurts TabPFN-2.6[1] and TabICLv2 while it helps models with a few million parameters, especially when a biomedical knowledge graph guides their feature attention [137].

A second group raises the limits of the released models. For wide tables, continued pretraining on synthetic data lets TabPFNv2 handle tens of thousands of features without losing accuracy on ordinary tables [138], and a frozen backbone can instead pool correlated neighboring features into meta-features [139]. Many classes are handled by decomposing the problem into binary subproblems through error-correcting output codes, whose design decides the accuracy [140], and a multi-target prior lets one forward pass predict several correlated targets while sharing information between them [141]. Text enters either through a small adapter that projects text embeddings into the token space of TabPFN, replacing the lossy PCA compression of textual columns [142], or as extra features, with row embeddings from a tabular language model that improve TabICLv2, TabPFN-2.6, and ConTextTab on semantically rich benchmarks [19].

A third group makes inference cheap. Distilling a PFN into gradient-boosted trees retains most of its accuracy at CPU latency [143], and exiting the forward pass early once predictions are confident exploits the depth redundancy of Section 4.2 at a small accuracy cost [144]. Quantizing the weights shrinks the checkpoint several-fold at similar accuracy [145], but since the checkpoints are small, the larger saving on big tables comes from quantizing the attention computation, which speeds up TabPFN-3 at almost no loss if training and test rows share the same quantized keys and values [146]. Layers can also be removed for a given dataset: replacing half of the layers of TabPFN-2.5 with small adapters trained on the target table costs almost no AUC [147]. However, which layers can go varies across datasets, and their effects do not add up: in TabICLv2, removing the individually least important layers costs more than removing random ones [148]. State-space layers give linear cost but read their input as a sequence: replacing the attention of the first TabPFN with them fits more context rows but loses accuracy [149], whereas SOMTab keeps attention for the in-context stage and approaches TabICLv2 with a much shorter runtime [150].

Training has also become more accessible. nanoTabPFN reimplements TabPFNv2 in a compact codebase that pretrains to useful accuracy in minutes on one GPU [65], and a public "speedrun" built on it has since cut that budget by almost two orders of magnitude [151]. Pretraining can also need far less data than expected: self-supervised pretraining on a single real table already transfers across domains, the value of a table depends on how many prediction tasks it supports rather than on its row count, and at corpus scale cleaning columns helps whereas deduplicating tables does not [92]. Finally, a model can spend more computation at inference time: compressing the hidden states of a first forward pass into a few extra tokens and running the model again improves prediction on most tabular datasets, and on average outperforms both a deeper model and a looped one [152].

5. Benchmarks

Model versions succeed each other within months (Section 3.1), so a comparison against last year's checkpoint is stale on publication. Benchmarks therefore divide into two kinds. Static suites such as OpenML-CC18 [153] and

[1] TabPFN-2.6 is an intermediate release between TabPFN-2.5 and TabPFN-3 that has no official technical report, so we do not count it among the core models.

Table 1
Benchmarks used to evaluate TFMs.

Benchmark	Measures	Tasks	Datasets
Static			
OpenML-CC18 [153]	Accuracy	Classification	72
TabZilla [6]	Accuracy	Classification	176
OmniTabBench [158]	Accuracy, dataset metafeatures	Classification, regression	3,030
Text suite [155]	Textual capabilities	Classification, regression	13
STRABLE [17]	String columns	Classification, regression	108
TEmBed [156]	Embedding quality	Prediction, semantics	69
Living			
TabArena [7]	Accuracy, tuning, ensembling	Classification, regression	51
BeyondArena [104]	Accuracy beyond IID	Classification, regression	142
ScoringBench [154]	Distributional predictions	Regression	97
TabBench-Bio [157]	Wide, small-sample tables	Classification, regression	43

TabZilla [6] fix their datasets and protocols at release. Living benchmarks revise datasets, implementations, and protocols as new models appear, so that their leaderboards track the current state of the field. The reference point of the second kind is TabArena, the first living benchmark for tabular prediction [7]. Its datasets are selected by a largely manual process that checks each candidate for task uniqueness, independence of the rows, provenance, leakage, and errors, discards the trivially solvable ones, and keeps a small fraction of the candidate pool. The launch study shows that the validation protocol and the ensembling of hyperparameter configurations change the rankings substantially, that gradient-boosted trees remain strong while deep models catch up under larger budgets with ensembling, that foundation models lead on small datasets, and that ensembles across model families set the state of the art. The same codebase now hosts BeyondArena, which extends the evaluation to temporal and grouped splits and to large and high-dimensional tables [104].

Six complementary benchmarks cover qualities that an accuracy leaderboard misses. ScoringBench evaluates distributional regression with proper scoring rules and shows that rankings shift substantially once the metric moves beyond point estimates, and even between different proper scoring rules [154]. A curated suite of datasets mixing structured and free-text columns measures textual capabilities, where no single strategy for incorporating the text dominates [155]. STRABLE widens the scope to 108 real tables with string columns of any kind, most of which hold short categorical strings rather than free text; there, simple string encoders feeding TabPFN-2.5 or TabICLv2 dominate the trade-off between accuracy and runtime, and the end-to-end semantic models TabSTAR and ConTextTab rank below the best modular pipelines [17]. TEmBed compares embeddings at the cell, row, column, and table level and finds that the best model depends on the task and the level: PFNs lead on prediction, whereas language-model encoders lead on semantic tasks such as row and column similarity search [156]. TabBench-Bio, a living benchmark, targets biomedical tables with thousands of features and few labeled rows on a grid of feature and sample budgets [157]. OmniTabBench instead varies scale, aggregating 3,030 datasets labeled by industry and metafeatures. It confirms that no family prevails overall and that the winner tracks dataset properties, with neural networks ahead on larger data with many categorical features, gradient-boosted trees on skewed and heavy-tailed features, and TabPFN on small datasets; it also argues that suites of a few dozen datasets are exposed to selection bias, since the ordering of models depends on which datasets were sampled [158]. Table 1 compares the benchmarks of this section.

6. Discussion and Future Directions

This section collects the open problems that the survey has exposed. Section 6.1 reconstructs the open challenges from the limitations that the frontier models state, read against the partial answers that the extension works already provide. Section 6.2 surveys the tasks beyond supervised prediction on a single table that TFMs are beginning to address, which fall outside our inclusion criteria. Section 6.3 closes with the limitations of this survey.

6.1. Open Challenges

Converting scale into accuracy. Each TabPFN generation has raised the size of the tables it is validated on: TabPFNv2 up to 10,000 samples and 500 features [2], TabPFN-2.5 up to 50,000 samples and 2,000 features [28], TabPFN-3 up to one million samples, within a fixed cell budget that trades rows against features [38], and TabPFN-3.5 to one million samples together with 6,000 features, its recommended limit [39]. Implementation advances let released checkpoints process longer contexts and wider tables as well (Sections 4.5 and 4.6). Fitting more data, however, does not translate into a proportional benefit. On large tables, TFMs lose most of their advantage over conventional methods: TabPFN-3 leads on average, but not significantly over gradient-boosted trees [38], and longer contexts and bagging narrow the difference without removing it [29]. Too much context can even hurt: Mitra-v2 loses accuracy on contexts much longer than those it was fine-tuned on [31]. A statistical analysis offers an explanation: as the context grows, additional rows reduce the variance of TFM predictions, whereas the bias persists unless the context is restricted to the neighborhood of each query [91]. The open challenge is therefore to make additional context reduce bias, which moves the scaling problem from inference engineering to architecture and pretraining design.

Semantic meaning in pretraining. Synthetic and real-table pretraining have different limitations. Models pretrained on synthetic priors learn from statistical structure alone: TabPFNv2 tells columns apart through randomized tokens with no semantic content [85], TabICLv2 lists the lack of native support for column names and text as a limitation shared across the family [37], and how to give these models semantic alignment is stated as an open question [29]. Models pretrained on real tables can use semantics, but suitable data are scarce. ConTextTab reaches its best validation scores with a fraction of its corpus and then plateaus; most of its tables are short and thin, and adding a few hundred longer and wider ones brings no significant improvement [29]. Added to a simple string encoding, the embeddings of TARTE improve prediction most on tables whose strings resemble those of its pretraining knowledge base [53], and TabSTAR documents public evaluation datasets appearing in the corpora on which the text encoders of semantic models are pretrained [50]. The language-model adaptations have the opposite problem: TabGemma reads textual features but matches an untuned random forest on numeric benchmarks, which its authors attribute to the tokenization of numbers [57], and MachineLearningLM maps every value to a bounded integer, which preserves order but coarsens magnitude [58]. Each family thus names as an open problem what another already does well. Real-TabPFN continues synthetic pretraining on curated real tables [27], and a small adapter feeds frozen text embeddings into TabPFN (Section 4.6), but no current model handles numbers and semantics equally well. On tables with string columns, simple string encodings fed to numeric in-context learners still outrank the semantic end-to-end models [17].

Empirical progress ahead of theory. Current TFMs are designed by experiment, with little theory to guide the choice of prior and architecture. The Mitra authors state that the principles of prior design are poorly understood [30], and the TabPFN authors list stronger theoretical foundations among their open directions [2]. TabICLv2 gives direct evidence that prior and architecture are coupled: its architecture performs poorly on the earlier TabICL prior, and the earlier architecture does not benefit from the more diverse new prior [37]. TabPFN-3 improves its prior by correcting failures as they are found, adding high-frequency functions and more realistic categorical variables to the generator after TabPFN-2.5 was seen to handle them poorly [38]. Mechanistic analyses expose inefficiencies that benchmark scores do not show: in the first layers of TabPFNv2, the hidden states collapse to a subspace of fewer than ten dimensions, so the model uses a small fraction of the capacity it pays for in parameters and compute [33]. Even which axis to scale is unclear: LimiX-2 finds the Elo rating that TabArena computes from pairwise comparisons growing log-linearly with the parameter count, whereas Mitra-v2 keeps a small model and reaches the level of the leading TFMs by broadening its synthetic pretraining tasks, concluding that model size is not the limiting factor [31, 34].

The extension works gather observations without a theory behind them. Diversity of generative mechanisms in the prior drives transfer, while similarity to real tables does not predict which datasets a model handles well and is only loosely associated with which prior yields the better model (Section 4.1); the same fine-tuning helps TabPFN and makes TabICL overfit (Section 4.6); and ensembling recovers almost nothing because current models make the same predictions on most inputs (Section 4.3). At present, the effect of a design choice is known only after pretraining. Whether a prior and an architecture can be successfully combined before training remains open.

Evaluation behind the models. Public benchmarks do not cover several capabilities that frontier models claim. When TabICLv2 was released, no established benchmark for distributional regression existed, so it evaluates this capability only on synthetic toy tasks [37]; ScoringBench has since filled this gap [154]. TabPFN-3 relies on internal benchmarks

for million-row prediction, many-class classification, and quantile regression: its many-class benchmark converts regression targets into classes, with only four naturally many-class datasets, and its quantile benchmark is adapted from TabArena regression tasks, since TabArena itself stops well short of a million rows [38]. Semantic evaluation is poor as well: ConTextTab reports that semantically informative benchmarks are scarce and that the CARTE benchmark suite consists mainly of small tables with binary classification targets [29]. The comparisons are confounded by mismatched hardware, tuning procedures, and the subsets of datasets each model supports [24, 28], and overlap between pretraining and evaluation datasets has been documented for some models [50]. How much the composition of the context matters is also understudied. TabPFN tolerates synthetic perturbations of its context (Section 4.3); a language model prompted with rows of the same table uses them inconsistently and loses accuracy when the context is noisy, and only recovers when trained to select a few evidence rows before predicting [159]; and context subsets of equal size give TabPFN-2.5 substantially different predictions [127]. The living benchmarks of Section 5 keep model comparisons up to date, but free-text semantics, context composition, and the internal consistency of predictions still lack standardized evaluations.

Narrowing access and documentation. Earlier releases such as TabPFNv2 combined public model access with an explicit assessment of limitations [2]. The leading edge of the field is increasingly shaped by industrial releases with incomplete access or technical documentation. TabPFN-3 withholds its strongest variant and restricts commercial use of the released model and its outputs [38], and TabPFN-3.5 continues this policy, offering its Plus and Thinking variants only through a commercial API [39]; EXAONE Tabular and Causilo likewise license their weights for non-commercial research only [35, 44]. At our review cutoff, TabFM and Nori are still documented only through a release page and a software deposit [15, 16]. Mitra-v2 is the exception among recent releases, with weights, inference and fine-tuning code, and per-dataset results under a permissive license, although without its pretraining pipeline [31]. A recent review that tabulates the licenses of current models finds the most capable ones at the restrictive end [14].

The licenses collected in Appendix C (Table C.2) show the same shift over time. Before 2025, only one core model restricted commercial use of its weights; in 2025, two of 16 did, and in 2026, seven of 13. Nearly all models still release at least one version of their weights, so access narrows mainly through licensing. Restrictions concentrate at the frontier: six of the seven leading entries on the live TabArena leaderboard carry non-commercial weights, with Mitra-v2 as the only exception [7]. This pattern limits independent assessment of model capabilities and makes it harder for researchers to reproduce, diagnose, and improve TFMs.

6.2. Applications Beyond Classification and Regression

Our corpus covers supervised classification and regression on a single static table, but the same checkpoints and the same pretraining procedures are already used well beyond that setting. This section, not included in the systematic review, gives an overview of these directions. Two main patterns arise across them. Most works reuse a released checkpoint unchanged and re-cast their task at its interface, through featurization, target transformations, or frozen embeddings. The rest pretrain a TFM on a synthetic prior built for the task.

Time series forecasting. A tabular checkpoint becomes a forecaster through feature engineering. TabPFN-TS predicts the value at each timestamp from lightweight temporal features with an unmodified TabPFNv2, turning forecasting into tabular regression [160], and TabPFN-3 releases a variant of it fine-tuned on synthetic time series for probabilistic forecasting [38], while the general TabPFN-3.5 checkpoint, with no time-series fine-tuning, places sixth of 29 methods on the fev-bench forecasting benchmark [39]. Other works pretrain forecasters entirely on series generated from an explicit temporal prior. ForecastPFN samples trend, seasonality, and noise components [161]; TimePFN moves to multivariate series drawn from Gaussian-process kernels [162]; ApolloPFN samples periodic signals and propagates them through structural causal models, so that the prior also covers series whose behavior changes with an external variable, and conditions predictions on the future values of that variable when they are known [163]. Mamba4Cast and TiRex keep the synthetic prior and replace the transformer with state-space and xLSTM backbones that scale better with the horizon [164, 165]. Other dedicated time-series foundation models are pretrained on large real-world corpora. Lag-Llama is a decoder-only transformer over lagged values [166] and TimesFM a patched decoder [167]; Chronos quantizes a series into a fixed vocabulary and treats it as a language [168]; Time-MoE is based on a mixture-of-experts architecture [169]; Sundial predicts next-patch distributions over continuous values through a flow-matching loss [170]. Time-LLM avoids pretraining a forecaster altogether, reprogramming a frozen language model by mapping series into text [171]. The two directions have since met. Unlike its predecessor, Chronos-2 is pretrained on synthetic data and shares information across series through group attention [172].

Data generation and imputation. A group of works builds data generators around released TFM checkpoints. TabPFGen reinterprets the discriminative outputs of a frozen TabPFN as an energy function and samples from it, without additional training or hyperparameter tuning, for augmentation, class balancing, and imputation [173]; TabEBM applies the same reinterpretation once per class, so that each class is sampled from its own energy landscape, which improves the quality of augmentations on small datasets [174]. TabImpute pretrains a TabPFN-style transformer that recasts imputation as supervised prediction over row–column context, on millions of synthetic tables produced by a generator with pluggable missingness modules [175]. Latent-space generators pretrain instead of reusing a checkpoint. CTSyn couples an autoencoder that projects heterogeneous tables into a shared latent space through table-schema embeddings with a conditional latent diffusion model that samples from that space and decodes back into table-specific values [176], and TabFORGE pairs a causality-aware feature encoder with a score-based diffusion transformer and a denoising-aligned decoder, reportedly beating dataset-specific generators in global utility across 45 datasets [177].

Survival analysis. Survival applications pair regression with censored data, where the observed time underestimates the true event time in the most common case of right censoring. A direct approach discretizes time into bins and casts survival analysis as a series of binary classifications, with censored patients entering as examples whose labels are missing beyond their follow-up, so that an off-the-shelf TFM predicts survival curves in context [178]. Two variants keep the model untouched and change the target instead: one replaces censored targets with jackknife pseudo-observations of the restricted mean survival time, turning survival into horizon-specific regression [179], and one builds an accelerated failure time model around a TFM, imputing censored times iteratively with an in-context version of the Buckley–James estimator [180]. Another strategy is to pretrain new PFNs or adapt released checkpoints on synthetic right-censored data-generating processes, approximating Bayesian posterior predictive inference [181–183].

Causal inference. The synthetic lineage already pretrains on structural causal models, so the prior can be redirected from prediction to causal quantities. Do-PFN samples interventions during pretraining and estimates interventional outcomes from observational data alone, with no access to the causal graph [184]. CausalFM sets out which SCM-based priors make Bayesian causal inference valid and instantiates them for back-door, front-door, and instrumental-variable adjustment [185], and FairPFN pretrains on synthetic causal fairness data to remove the causal effect of protected attributes without knowing the causal model [186]. PFN treatment-effect estimates can carry a prior-induced confounding bias, which a calibration step removes [187]. Causal structure also survives in models never trained for it, since adjacency matrices can be decoded from the frozen embeddings of a predictive TabPFN [188], and LimiX-2 recovers causal skeletons from its feature attention more accurately than models that compress each row before in-context learning [34].

Bayesian inference and decision-making. The PFN was formulated as approximate Bayesian inference before it became a tabular predictor, and a group of works returns to that formulation [11]. TabPFN serves as a pretrained conditional density estimator for simulation-based inference, matching specialized methods at far smaller simulation budgets [189], and Cluster-PFN pretrains on a Gaussian-mixture prior for Bayesian clustering [190]. In sequential decisions, a PFN estimates the posterior that drives the next choice: as a surrogate in Bayesian optimization [191, 192], as a learning-curve extrapolator that stops unpromising runs early and guides freeze-thaw hyperparameter optimization [193, 194], as the sampling distribution for Thompson sampling in contextual bandits [195], and as an in-context regressor of value functions that solves control tasks with no gradient updates [196, 197].

Graphs and relational databases. Data that spans linked entities can be flattened into one table, or handled natively by a model that reads the links. Graph learning takes the first option by making each node a row: G2T-FM augments node features with neighborhood aggregations and structural embeddings before passing them to TabPFNv2 or LimiX [198, 199]. GraphPFN takes the second, adding neighborhood aggregation layers to LimiX and pretraining on millions of synthetic attributed graphs [200], and in-context learners reach link prediction and knowledge-graph reasoning as well [201, 202]. Databases with multiple linked tables split the same way. RDBLearn featurizes each target row through automatic relational aggregations and runs an off-the-shelf TFM on the resulting single table [203], and TabPFN-Rel refines this interface for TabPFN-3 [204]. RDB-PFN pretrains on an unbounded stream of synthetic relational databases [205], Relational Transformer attends over columns, rows, and primary–foreign key links, and predicts on unseen schemas with no task-specific fitting [206], Griffin follows a graph-centric route [207], and KumoRFM-2 reads connected tables directly, without flattening or manual target construction [208].

Multimodal data. The simplest strategy to make a TFM multimodal is to treat the outputs of a pretrained encoder as feature columns. Reducing DINOv3 image features to 500 principal components lets TabPFN classify AI-generated images from a handful of labeled examples [209], and a study across image, text, and audio encoders finds TabPFN the best-calibrated of nine few-shot heads [210]. Other works change the architecture instead of the inputs. TIME uses a frozen TabPFN as a tabular encoder and fuses its embeddings with image backbones, which keeps predictions accurate when tabular values are missing [211], and MultiModalPFN trains modality projectors that map image and text embeddings into TabPFN's token space, so that its pretrained prior applies to all modalities at once [212].

6.3. Limitations of This Survey

Our scope can be read as too narrow. Works that study the TFM approach beyond supervised prediction on a single static table fall outside the corpus, and Section 6.2 discusses them only informally, without the guarantees of the systematic review. In a young, fast-moving field, we chose to give a systematic treatment only to a focused core, and we drew the boundary around the works that define it.

The opposite reading is equally fair. Figure 3 shows a corpus split into two loosely connected groups, PFNs pretrained on synthetic priors and fine-tuned models pretrained on real tables, and a stricter definition would retain only one of them. We keep both because they pursue the same goal, a model pretrained once and reused across tables. Reading them together is what exposes their complementary limitations in Section 6.1.

Any snapshot of this field also ages quickly. The corpus grew by 80 papers between January and mid-September 2026 (Section 2.3), and our search discards everything published after 15 September 2026. Rather than claiming durability, we designed the survey to be extended: the two axes of Section 3 give a new model its coordinates and dependencies, the threads of Section 4 give a new study a place to attach, and the living benchmarks of Section 5 keep track of the frontier.

A final limitation concerns how the corpus was built. The screening delegates the first judgment to two language models working from titles and abstracts, and we read the full texts of the included papers and of the records whose title and abstract were not conclusive; of the 769 unanimous exclusions, we audited 216 by hand. Every audited exclusion was confirmed and the two screeners agree almost perfectly ($\kappa = 0.822$), but a relevant work rejected by both screeners outside the audit would be lost without notice. A major caveat is that nearly two thirds of the corpus circulates as preprints at the time of writing, and two frontier models are documented by a blog post or a repository alone (Section 6.1). We assessed the quality of each inclusion to the best of our ability, but part of the record this survey rests on has not passed peer review.

7. Conclusion

We presented a systematic survey of tabular foundation models, screening 1,133 candidate records with two language models and adjudicating the results by hand, obtaining a curated corpus of 146 papers. We organized the corpus along two axes, the pretraining data source and the transfer mechanism, and placed core models and their dependencies on them. This revealed a single lineage descending from TabPFN, on which most industrial releases now build, next to smaller families pretrained on real tables. We organized the 104 extension works into six threads, from designing the prior to adapting and scaling, reviewed the benchmarks that keep the comparison of frontier models current, and reconstructed the open challenges from the stated limitations. Beyond the corpus, we mapped the tasks that the same pretrained models already serve, from forecasting and survival analysis to causal inference and relational data. The result is a reference for what is known about tabular foundation models and for what is yet to be discovered. The open challenges name the problems that future TFMs will be tested on.

CRediT authorship contribution statement

Alberto Archetti: Conceptualization, Data curation, Investigation, Methodology, Project administration, Software, Supervision, Visualization, Writing – original draft, Writing – review & editing. **Marina Mastroleo:** Conceptualization, Investigation, Validation, Writing – original draft, Writing – review & editing. **Mattia Sabella:** Conceptualization, Investigation, Validation, Writing – original draft, Writing – review & editing. **Cinzia Cappiello:** Conceptualization, Project administration, Supervision, Writing – review & editing. **Matteo Matteucci:** Conceptualization, Project administration, Supervision, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Data availability

The search results, screening outputs, extracted model characteristics, and analysis code supporting this survey are available at `https://github.com/archettialberto/tfm-survey`.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Claude (Fable 5, Fable 5.1, Opus 5, and Opus 5.5; Anthropic) and ChatGPT (5.6 Sol and 6 Astra; OpenAI) in order to organize and review the text. After using these tools, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article. The use of language models for record screening is part of the review methodology and is described in Section 2.2.

Table A.1
Keyword ablation for the three selected search terms (top) and the discarded candidates retrieving at least ten records (bottom).

Search Term	Records	New Records	New Papers
TabPFN	832	442	17
tabular foundation model	399	208	53
prior-data fitted network	334	89	6
TabICL	59	9	0
foundation model for tabular data	40	11	2
LimiX	32	22	0
tabular in-context learning	26	4	1
tabular language model	17	17	1
TabDPT	13	0	0
Mitra (with "tabular")	13	5	0
TabFM	12	2	0
TabLLM	10	8	1

Records counts the unique records a term retrieves from both bibliographic sources. **New Records** counts those not retrieved by the selected terms (for a selected term, by the other two), and **New Papers** counts how many of them belong to the 146-paper corpus.

A. Keyword Set Ablation

The keyword set of Section 2.1 was selected from 35 candidate terms, spanning alternative phrasings of "tabular foundation model", names of individual models, and, for ambiguous names, the model name paired with required context words. Each candidate was queried on both bibliographic sources under the same 15 September 2026 cutoff. Table A.1 reports how many unique records each term retrieves and what it adds on top of the selected terms, both as raw records and as papers of the final corpus.

The three selected terms cover 131 of the 146 papers in the corpus. Dropping "TabPFN", "tabular foundation model", or "prior-data fitted network" loses 17, 53, and 6 papers, respectively. The 32 discarded candidates together add 98 records. Six of them are corpus papers, all among the manual additions. We screened the remaining 92 with the two models of Section 2.2, and reviewed by hand every record they did not unanimously exclude; all eligible works among them are included as manual additions.

B. Screening Prompt

Each screening request pairs a fixed system prompt with a user message that concatenates the task instructions, the scope statement, the one-line definitions of every role and category, and the record's title, year, authors, venue, DOI, and abstract serialized as JSON. The system prompt reads:

```
Screen scientific records for an architectural survey of tabular foundation models using only the
supplied title and abstract. Use explicit evidence, do not infer unstated content, and return
unclear whenever a requested judgment cannot be resolved. Return the requested JSON.
```

The task instructions read:

```
First give one evidence-based reason of at most 35 words, then return two judgments.

1. category: First apply the task boundary. Use out_of_scope for an excluded model or final task
even if the paper proposes a novel method. Otherwise select the paper's primary contribution.
2. tfm_role: Judge what the work does with TFMs.

A claimed reusable contribution transferable beyond one domain is primary_contribution;
domain-specific use without one is primary_application_method. For surveys, explicit TFM
discussion or comparison is primary_object_of_study even when the survey is broader; a bare name
or citation is baseline_or_mention.
```

```
Use unclear for either field when the title and abstract do not support a decision. If the
abstract is absent, use explicit title evidence only.
```

The scope statement defines the boundary of the survey:

```
A tabular foundation model (TFM) is pretrained or meta-trained across datasets or tasks and
intended for reuse on new tabular datasets, with or without adaptation. The review covers reusable
TFM work for ordinary tabular classification or regression: models, architecture, pretraining,
data or prior synthesis and selection, scaling, efficiency, adaptation, postprocessing,
ensembling, analysis, benchmarks, tooling, and surveys. Generation, cleaning, imputation, and
feature selection qualify only when used for TFM pretraining or to support downstream
classification or regression; they are out of scope as final tasks. Also exclude forecasting and
other sequence-dependent tasks, survival analysis, anomaly detection, clustering, causal
inference, Bayesian optimization, graph or knowledge-graph tasks, relational learning, multimodal
prediction, and table understanding such as question answering, text-to-SQL, extraction, or
spreadsheet parsing. Data derived from another modality qualify only when the model consumes an
ordinary table and does not use the original temporal, relational, or multimodal structure.
Conventional per-dataset models and foundation models for other modalities are out of scope.
```

The role definitions read:

```
- primary_contribution: The work builds or changes a TFM itself, or its reusable training,
adaptation, or inference method, so the result carries over to other datasets.
- primary_object_of_study: The work analyzes, benchmarks, or provides tooling for TFMs; for a
survey, any explicit TFM discussion, comparison, or classification qualifies even if the survey is
broader.
- primary_application_method: An existing TFM is a principal evaluated method for one domain
problem without a reusable TFM contribution.
- baseline_or_mention: In non-survey work, a TFM appears only as a baseline, secondary comparator,
or incidental mention; in a survey, it is only named or cited without discussion.
- absent: No model meeting the TFM definition is present.
- unclear: The title and abstract do not establish the TFM's role.
```

The category definitions read:

```
- new_model: Introduces a new TFM or cross-dataset pretraining method.
- model_extension: Contributes a reusable TFM change, such as architecture, pretraining, data or
prior design, objectives, adaptation, inference, postprocessing, distillation, ensembling, or
efficiency.
- analysis: Produces generalizable findings or theory about TFMs, not merely results or
interpretation from one domain application.
- benchmark: Contributes a reusable TFM-focused dataset, protocol, or broad comparative evaluation.
- application: Uses existing TFMs for a specific domain problem without a claimed reusable TFM
contribution or general TFM analysis.
- tooling: Contributes reusable TFM software or infrastructure.
- survey: A survey explicitly discusses, compares, or classifies TFMs, even if TFMs are not its
main topic.
- out_of_scope: The model or final task does not meet the scope definition; this takes precedence
over all contribution categories.
- unclear: The available title and abstract do not establish a contribution category.
```

C. Core Model Characteristics

Tables C.1 and C.2 summarize the 42 core models of Section 3.1. Table C.1 reports the developer, first release, size, supported inputs and tasks, and publication venue of each model, and Table C.2 reports how the model can be obtained and under which licenses. We collected the values from the introducing papers, the official repositories and model cards, the license files, and the documentation of the developers, all accessed in September 2026. The limits on

rows, features, and classes are those that developers document or recommend for the released model. Dates are first public releases and can precede the publication dates used in Figure 2.

Table C.1: Characteristics of the 42 core models, sorted by release date.

Model	Organization	Date	Parameters	Rows	Features	Classes	Tasks	Inputs	Adaptation	Venue
PFN [4]	Univ. Freiburg	2021-12	–	–	–	–	C R	N	In-context	ICLR
TabPFN [5]	Univ. Freiburg	2022-07	26M	1k	100	10	C	N C M	In-context	ICLR
XTab [46]	Amazon, EPFL, Cornell	2023-05	<1M	–	–	none	C R	N C	Fine-tuning	ICML
UniTabE [47]	HKU, BAAI	2023-07	–	–	–	none	C R I Z	N C T	Fine-tuning	ICLR
LLM-GTL [55]	Microsoft, Tsinghua Univ.	2023-10	7B / 13B	–	–	none	C R Z	N C T	In-context	KDD
MotherNet [59]	Microsoft	2023-12	89M	5k	100	10	C	N	Hypernetwork	ICLR
CARTE [51]	Inria	2024-02	9.3M	–	–	none	C R	N C T M	Fine-tuning	ICML
HyperFast [60]	Stanford, UPC	2024-02	1.27B	–	none	100	C	N C	Hypernetwork	AAAI
TP-BERTa [49]	Zhejiang Univ.	2024-03	125M†	–	–	none	C R	N C T	Fine-tuning	ICLR
TabForest [22]	KAIST	2024-05	39M†	–	100*	10	C	N	In-context, fine-tuning	Preprint
TabuLa-8B [56]	Univ. Washington, Stanford	2024-06	8B	–	–	none	C R Z	N C T M	In-context	NeurIPS
PORTAL [48]	SAP	2024-10	89M†	–	–	none	C R	N C T D M	Fine-tuning	NeurIPS Workshop
TabDPT [23]	Layer 6 (TD Bank)	2024-10	78M	–	100*	10*	C R	N C	In-context	NeurIPS
TabPFNv2 [2]	Prior Labs, Univ. Freiburg	2025-01	7M / 11M	10k	500	10	C R D G E O	N C M	In-context, fine-tuning	Nature
TabICL [36]	Inria	2025-02	27M†	500k	500	10*	C	N C	In-context	ICML
TARTE [53]	Inria	2025-05	25M	–	–	none	C R E	N C T D M	Fine-tuning	TMLR
TabSTAR [50]	Technion	2025-05	47M†	–	–	none	C R E	N C T D M	Fine-tuning	NeurIPS
ConTextTab [29]	SAP	2025-06	16M	8k	500	62	C R	N C T D M	In-context	NeurIPS
TabFlex [25]	Microsoft, UW–Madison	2025-06	–	–	1,000*	100	C	N	In-context	ICML
Mitra [30]	Amazon	2025-07	77M	10k	500	10	C R	N C	In-context, fine-tuning	NeurIPS
Real-TabPFN [27]	Univ. Freiburg, Prior Labs	2025-07	7M	10k	500	10	C	N C M	In-context	ICML Workshop
LimiX [32]	Stable AI, Tsinghua Univ.	2025-08	16M	–	–	10	C R I G E	N C M	In-context	Preprint
MachineLearningLM [58]	UCAS, Microsoft, Stanford	2025-09	7B	1,024	50	10	C	N C T	In-context	Preprint
LimiX-2M [33]	Stable AI, Tsinghua Univ.	2025-11	1.9M	–	–	10	C R I	N C M	In-context	ICML
Orion-Bix [40]	Lexsi Labs	2025-11	27M†	–	–	10*	C	N C	In-context	WWW
Orion-MSP [41]	Lexsi Labs	2025-11	44M†	–	–	10*	C	N C	In-context	Preprint
TabGemma [57]	SAP	2025-11	12B	–	–	none	C R	N C T D	In-context	EurIPS Workshop
TabPFN-2.5 [28]	Prior Labs	2025-11	11M / 10M	50k	2,000	10	C R	N C M	In-context, fine-tuning	Preprint
iLTM [61]	Stanford, UC Santa Cruz	2025-11	–	–	none	100	C R	N C M	Hypernetwork, fine-tuning	KDD
TabICLv2 [37]	Inria	2026-02	28M	1M	2,000	10*	C R F	N C	In-context, fine-tuning	ICML
TabH2O [42]	H2O.ai	2026-05	29M	500k	100	10*	C R F I O E	N C	In-context	Preprint
TabPFN-3 [38]	Prior Labs	2026-05	53M / 58M	1M	2,000	160	C R F E	N C M	In-context, fine-tuning	Preprint
Nori [16]	Synthefy	2026-06	6M	50k	1,000	n/a	R F	N C T M	In-context	Release note
TabDPT-Turbo [24]	Layer 6 (TD Bank), Cohere	2026-06	64M†	100k	128	16	C R	N C	In-context	ICML Workshop
TabFM [15]	Google Research	2026-06	1.6B†	–	500	10	C R	N C D	In-context	Release note
TabSwift [26]	Nanjing Univ.	2026-06	8M†	–	100*	10*	C R	N C	In-context	ICML
EXAONE Tabular [35]	LG AI Research	2026-08	21M	100k*	100 / 1,024*	10*	C R	N C M	In-context	Preprint
Causilo [44]	Nums AI	2026-09	36M	–	–	10*	C R	N C M	In-context	Preprint
LimiX-2 [34]	Stable AI, Tsinghua Univ.	2026-09	406M	–	–	10	C R I	N C D M	In-context	Preprint
Mitra-v2 [31]	Amazon	2026-09	77M	32k	256*	10*	C R	N C	In-context, fine-tuning	Preprint
TabPFN-3.5 [39]	Prior Labs	2026-09	220M	1M	6,000	160	C R F	N C D M	In-context, fine-tuning	Preprint
Xiaomi-TabLDM [43]	Xiaomi	2026-09	70M‡	–	–	10*	C R	N C	In-context	Preprint

Date: first public release. **Parameters**, **Features**: two values refer to separate classification and regression checkpoints; † computed from the released checkpoint; ‡ mixture of experts with 62M active parameters. **Rows**, **Features**, **Classes**: documented or recommended limits; * larger inputs are handled by a built-in reduction or class decomposition; none, no architectural limit; –, not documented; n/a, not applicable. **Tasks**: classification (C), regression (R), forecasting (F), imputation (I), generation (G), density estimation (D), embeddings (E), outlier detection (O), zero-shot prediction without labeled rows (Z). **Inputs**: numerical (N), categorical (C), free text (T), and date (D) features, and native handling of missing values (M). **Adaptation**: how the pretrained model is applied to a new dataset.

Table C.2: Availability of the 42 core models, in the order of Table C.1.

Model	Code	Weights	Use	API	Integrations	GitHub	Hugging Face
PFN	Apache-2.0	Apache-2.0	Unrestricted	–	–	Link	–
TabPFN	Apache-2.0	Apache-2.0	Unrestricted	–	Package, AutoGluon	Link	–
XTab	None	None	Unclear	–	–	Link	–
UniTabE	Apache-2.0	–	–	–	–	Link	–
LLM-GTL	MIT	MIT + Llama 2	Conditional	–	–	Link	Link
MotherNet	Apache-2.0	Apache-2.0	Unrestricted	–	–	Link	–
CARTE	BSD-3	BSD-3	Unrestricted	–	Package	Link	–
HyperFast	CC-BY-NC-4.0	CC-BY-NC-4.0	Research only	–	Package	Link	–
TP-BERTa	MIT	MIT	Unrestricted	–	–	Link	–
TabForest	MIT	MIT	Unrestricted	–	–	Link	Link
TabuLa-8B	MIT	Llama 3	Conditional	–	–	Link	Link
PORTAL	Apache-2.0	Apache-2.0	Unrestricted	–	–	Link	–
TabDPT	Apache-2.0	Apache-2.0	Unrestricted	–	Package, TabArena, TabTune	Link	Link
TabPFNv2	Apache-2.0	Prior Labs v1.1	Unrestricted	Prior Labs	Package, AutoGluon, TabArena	Link	Link
TabICL	BSD-3	BSD-3	Unrestricted	–	Package, AutoGluon, TabArena, TabTune	Link	Link
TARTE	BSD-3	MIT	Unrestricted	–	Package	Link	Link
TabSTAR	MIT	CC-BY-4.0	Unrestricted	–	Package, TabArena	Link	Link
ConTextTab	Apache-2.0	Apache-2.0[§]	Research only	SAP	Package, TabArena, TabTune	Link	Link
TabFlex	Apache-2.0	Apache-2.0	Unrestricted	–	–	Link	–
Mitra	Apache-2.0	Apache-2.0	Unrestricted	–	AutoGluon, TabArena, TabTune	Link	Link
Real-TabPFN	Apache-2.0	Prior Labs v1.1	Unrestricted	Prior Labs	Package, AutoGluon, TabArena	Link	Link
LimiX	Stable AI v1.0	Stable AI v1.0	Unrestricted	–	TabArena, TabTune	Link	Link
MachineLearningLM	MIT	Apache-2.0	Unrestricted	–	–	Link	Link
LimiX-2M	Stable AI v1.0	Stable AI v1.0	Unrestricted	–	–	Link	Link
Orion-Bix	MIT	MIT	Unrestricted	–	Package, TabTune	Link	Link
Orion-MSP	MIT	MIT	Unrestricted	–	Package, TabArena, TabTune	Link	Link
TabGemma	–	–	–	–	–	–	–
TabPFN-2.5	Apache-2.0	Non-commercial	Research only	Prior Labs	Package, AutoGluon, TabArena	Link	Link
iLTM	Apache-2.0	Apache-2.0	Unrestricted	–	Package, TabArena	Link	Link
TabICLv2	BSD-3	BSD-3	Unrestricted	–	Package, AutoGluon, TabArena, TabTune	Link	Link
TabH2O	Apache-2.0[¶]	–	Via API	H2O.ai	–	Link	–
TabPFN-3	Apache-2.0	Non-commercial	Research only	Prior Labs	Package, AutoGluon, TabArena	Link	Link
Nori	Apache-2.0	Apache-2.0	Unrestricted	Synthefy	Package, TabArena	Link	Link
TabDPT-Turbo	Apache-2.0	Apache-2.0	Unrestricted	–	Package, TabArena	Link	Link
TabFM	Apache-2.0	Non-commercial	Research only	BigQuery	Package, TabArena, TabTune	Link	Link
TabSwift	MIT	Apache-2.0	Unrestricted	–	–	Link	Link
EXAONE Tabular	BSD-3 (LG)	Non-commercial	Research only	–	Package, TabArena	Link	Link
Causilo	Apache-2.0	Non-commercial	Research only	–	Package	Link	Link
LimiX-2	Stable AI v1.0	Non-commercial	Research only	–	TabArena	Link	Link
Mitra-v2	Apache-2.0	Apache-2.0	Unrestricted	–	AutoGluon, TabArena	–	Link
TabPFN-3.5	Apache-2.0	Non-commercial	Research only	Prior Labs	Package, AutoGluon, TabArena	Link	Link
Xiaomi-TabLDM	Apache-2.0	Apache-2.0	Unrestricted	–	Package	Link	Link

Code, **Weights**: license of the official code and of the released checkpoints; –, not released; None, released without a license. Prior Labs v1.1 is Apache-2.0 with an attribution clause, and Stable AI v1.0 is derived from Apache-2.0 with attribution and naming requirements. [§] Declared Apache-2.0, but restricted to research by the license of the pretraining data. [¶] API client only. **Use**: what the license of the released model permits; unrestricted, research and commercial use; research only, commercial use only through a separate license or a service of the developer, where one exists; conditional, commercial use subject to the license of the base language model; via API, any use only through the hosted API; unclear, no license stated. **API**: hosted service of the developer. **Integrations**: dedicated scikit-learn-compatible package and support in AutoGluon, TabArena, TabTune, or TALENT. **GitHub**, **Hugging Face**: link to the repository.

D. PRISMA Checklist

This appendix reports the PRISMA 2020 checklist [8] for this survey. Section D.1 gives additional information required for the checklist, and Section D.2 maps each item to the place where it is addressed.

D.1. Additional Information

Abstract. Item 2 of the checklist points to the separate PRISMA 2020 for Abstracts checklist, which our abstract follows in part. It does not state the eligibility criteria, the bibliographic sources and the search date, or the limitations of the evidence, which are reported in Sections 1, 2.1, and 6.3 instead. It also carries no statement of funding or registration, which do not fit within the 250-word limit of the journal.

Registration and protocol. This review was not registered in any register of systematic reviews, and no protocol was prepared before the search was executed. Therefore, Item 24c of the checklist does not apply.

Support and competing interests. This review received no specific financial or non-financial support. The authors declare no competing interests.

Synthesis method. We synthesize the corpus narratively. The included works are methodological contributions: each reports accuracy on its own selection of datasets, under evaluation protocols that the field is still standardizing (Section 5). No effect measure is therefore comparable across the corpus. Instead, we assign each paper to one group of the taxonomy of Section 3 or to one of the six threads of Section 4, and describe the groups in prose.

Items that do not apply. Several checklist items assume a meta-analysis of quantitative outcomes and have no counterpart here. Because we pool no results, we define no effect measure (item 12), convert no summary statistics (13b), explore no statistical heterogeneity (13e, 20c), and run no sensitivity analyses (13f, 20d). We also present no per-study effect estimates with their precision (19). We assess neither the risk of bias of the included works (11, 18) nor the certainty of a body of evidence (15, 22). For the same reason we do not assess bias arising from missing results (14, 21). What we do report about the quality of the record is the publication status of the corpus, nearly two thirds of which circulates only as preprints, and the incomplete documentation of two frontier models, both discussed in Section 6.3. The characteristics of the works contributing to each group are summarized in Sections 3 and 4, without an accompanying risk-of-bias summary (20a).

D.2. Checklist Table

Table D.1 lists the 27 checklist items and, for each one, where it is addressed.

Table D.1: PRISMA 2020 checklist for this systematic survey.

Section and Topic	Item #	Checklist item	Location where item is reported
TITLE			
Title	1	Identify the report as a systematic review.	Title
ABSTRACT			
Abstract	2	See the PRISMA 2020 for Abstracts checklist.	Abstract; Section D.1
INTRODUCTION			
Rationale	3	Describe the rationale for the review in the context of existing knowledge.	Section 1
Objectives	4	Provide an explicit statement of the objective(s) or question(s) the review addresses.	Section 1
METHODS			
Eligibility criteria	5	Specify the inclusion and exclusion criteria for the review and how studies were grouped for the syntheses.	Sections 1 and 2.2; Appendix B
Information sources	6	Specify all databases, registers, websites, organisations, reference lists and other sources searched or consulted to identify studies. Specify the date when each source was last searched or consulted.	Section 2.1
Search strategy	7	Present the full search strategies for all databases, registers and websites, including any filters and limits used.	Section 2.1; Appendix A
Selection process	8	Specify the methods used to decide whether a study met the inclusion criteria of the review, including how many reviewers screened each record and each report retrieved, whether they worked independently, and if applicable, details of automation tools used in the process.	Section 2.2; Figure 1; Appendix B
Data collection process	9	Specify the methods used to collect data from reports, including how many reviewers collected data from each report, whether they worked independently, any processes for obtaining or confirming data from study investigators, and if applicable, details of automation tools used in the process.	Sections 2.1 and 2.2; Appendix C
Data items	10a	List and define all outcomes for which data were sought. Specify whether all results that were compatible with each outcome domain in each study were sought (e.g. for all measures, time points, analyses), and if not, the methods used to decide which results to collect.	Sections 2.3 and 3
	10b	List and define all other variables for which data were sought (e.g. participant and intervention characteristics, funding sources). Describe any assumptions made about any missing or unclear information.	Section 2.1
Study risk of bias assessment	11	Specify the methods used to assess risk of bias in the included studies, including details of the tool(s) used, how many reviewers assessed each study and whether they worked independently, and if applicable, details of automation tools used in the process.	Section D.1
Effect measures	12	Specify for each outcome the effect measure(s) (e.g. risk ratio, mean difference) used in the synthesis or presentation of results.	Section D.1

continued on next page

Table D.1 (continued)

Section and Topic	Item #	Checklist item	Location where item is reported
Synthesis methods	13a	Describe the processes used to decide which studies were eligible for each synthesis (e.g. tabulating the study intervention characteristics and comparing against the planned groups for each synthesis (item #5)).	Sections 2.3, 3, and 4
	13b	Describe any methods required to prepare the data for presentation or synthesis, such as handling of missing summary statistics, or data conversions.	Section D.1
	13c	Describe any methods used to tabulate or visually display results of individual studies and syntheses.	Section D.1
	13d	Describe any methods used to synthesize results and provide a rationale for the choice(s). If meta-analysis was performed, describe the model(s), method(s) to identify the presence and extent of statistical heterogeneity, and software package(s) used.	Section D.1
	13e	Describe any methods used to explore possible causes of heterogeneity among study results (e.g. subgroup analysis, meta-regression).	Section D.1
	13f	Describe any sensitivity analyses conducted to assess robustness of the synthesized results.	Section D.1
Reporting bias assessment	14	Describe any methods used to assess risk of bias due to missing results in a synthesis (arising from reporting biases).	Section D.1
Certainty assessment	15	Describe any methods used to assess certainty (or confidence) in the body of evidence for an outcome.	Section D.1
RESULTS			
Study selection	16a	Describe the results of the search and selection process, from the number of records identified in the search to the number of studies included in the review, ideally using a flow diagram.	Section 2.2; Figure 1
	16b	Cite studies that might appear to meet the inclusion criteria, but which were excluded, and explain why they were excluded.	Sections 6.2 and 6.3
Study characteristics	17	Cite each included study and present its characteristics.	Sections 3 and 4
Risk of bias in studies	18	Present assessments of risk of bias for each included study.	Section D.1
Results of individual studies	19	For all outcomes, present, for each study: (a) summary statistics for each group (where appropriate) and (b) an effect estimate and its precision (e.g. confidence/credible interval), ideally using structured tables or plots.	Section D.1
Results of syntheses	20a	For each synthesis, briefly summarise the characteristics and risk of bias among contributing studies.	Section D.1
	20b	Present results of all statistical syntheses conducted. If meta-analysis was done, present for each the summary estimate and its precision (e.g. confidence/credible interval) and measures of statistical heterogeneity. If comparing groups, describe the direction of the effect.	Sections 3, 4, and 5
	20c	Present results of all investigations of possible causes of heterogeneity among study results.	Section D.1
	20d	Present results of all sensitivity analyses conducted to assess the robustness of the synthesized results.	Section D.1
Reporting biases	21	Present assessments of risk of bias due to missing results (arising from reporting biases) for each synthesis assessed.	Section D.1
Certainty of evidence	22	Present assessments of certainty (or confidence) in the body of evidence for each outcome assessed.	Section D.1
DISCUSSION			
Discussion	23a	Provide a general interpretation of the results in the context of other evidence.	Section 6.1
	23b	Discuss any limitations of the evidence included in the review.	Section 6.3
	23c	Discuss any limitations of the review processes used.	Section 6.3
	23d	Discuss implications of the results for practice, policy, and future research.	Sections 6.1 and 7
OTHER INFORMATION			
Registration and protocol	24a	Provide registration information for the review, including register name and registration number, or state that the review was not registered.	Section D.1
	24b	Indicate where the review protocol can be accessed, or state that a protocol was not prepared.	Section D.1
	24c	Describe and explain any amendments to information provided at registration or in the protocol.	Section D.1
Support	25	Describe sources of financial or non-financial support for the review, and the role of the funders or sponsors in the review.	Section D.1; Funding

continued on next page

Table D.1 (continued)

Section and Topic	Item #	Checklist item	Location where item is reported
Competing interests	26	Declare any competing interests of review authors.	Section D.1; Declaration of competing interest
Availability of data, code and other materials	27	Report which of the following are publicly available and where they can be found: template data collection forms; data extracted from included studies; data used for all analyses; analytic code; any other materials used in the review.	Appendix B; Data availability

References

[1] B. Van Breugel, M. Van Der Schaar, Position: why tabular foundation models should be a research priority, in: Proceedings of the 41st International Conference on Machine Learning, ICML'24, JMLR.org, 2024.

[2] N. Hollmann, S. Müller, L. Purucker, A. Krishnakumar, M. Körfer, S. B. Hoo, R. T. Schirrmeister, F. Hutter, Accurate predictions on small data with a tabular foundation model, Nature 637 (8045) (2025) 319–326.

[3] L. Grinsztajn, E. Oyallon, G. Varoquaux, Why do tree-based models still outperform deep learning on typical tabular data?, Advances in Neural Information Processing Systems 35 (2022) 507–520.

[4] S. Müller, N. Hollmann, S. Pineda Arango, J. Grabocka, F. Hutter, Transformers Can Do Bayesian Inference, in: International Conference on Learning Representations, 2022.
URL https://openreview.net/forum?id=KSugKcbNf9

[5] N. Hollmann, S. Müller, K. Eggensperger, F. Hutter, TabPFN: A Transformer That Solves Small Tabular Classification Problems in a Second, in: International Conference on Learning Representations, 2023.
URL https://openreview.net/forum?id=cp5PvcI6w8_

[6] D. McElfresh, S. Khandagale, J. Valverde, V. Prasad C, G. Ramakrishnan, M. Goldblum, C. White, When do neural nets outperform boosted trees on tabular data?, Advances in Neural Information Processing Systems 36 (2023) 76336–76369.

[7] N. Erickson, L. Purucker, A. Tschalzev, D. Holzmüller, P. Desai, D. Salinas, F. Hutter, TabArena: A living benchmark for machine learning on tabular data, Advances in Neural Information Processing Systems 38 (2025).

[8] M. J. Page, J. E. McKenzie, P. M. Bossuyt, I. Boutron, T. C. Hoffmann, C. D. Mulrow, L. Shamseer, J. M. Tetzlaff, E. A. Akl, S. E. Brennan, R. Chou, J. Glanville, J. M. Grimshaw, A. Hróbjartsson, M. M. Lalu, T. Li, E. W. Loder, E. Mayo-Wilson, S. McDonald, L. A. McGuinness, L. A. Stewart, J. Thomas, A. C. Tricco, V. A. Welch, P. Whiting, D. Moher, The PRISMA 2020 statement: an updated guideline for reporting systematic reviews, BMJ 372 (2021).

[9] S. Somvanshi, S. Das, S. A. Javed, G. Antariksa, A. Hossain, A survey on deep tabular learning, arXiv preprint arXiv:2410.12034 (2024).

[10] J.-P. Jiang, S.-Y. Liu, H.-R. Cai, Q.-L. Zhou, H.-J. Ye, Representation learning for tabular data: A comprehensive survey, IEEE Transactions on Pattern Analysis and Machine Intelligence (2026).

[11] S. Müller, A. Reuter, N. Hollmann, D. Rügamer, F. Hutter, The future of Bayesian prediction is prior-fitted, in: Proceedings of the 42nd International Conference on Machine Learning, ICML'25, JMLR.org, 2025.

[12] T. Cong, J. M. Eisenschlos, D. Gomm, L. Grinsztajn, A. C. Müller, A. Sanghi, J.-M. Bodensohn, V. Borisov, M. Cochez, K. Eggensperger, F. Geerts, M. J. Kim, A. Kipf, X. Li, O. Ovcharenko, P. Papotti, L. Purucker, S. Schelter, I. Trummer, G. Varoquaux, L. Vogel, C. Binnig, M. Hulsebos, F. Hutter, Unlocking the Full Potential of Data Science Requires Tabular Foundation Models, Agents, and Humans, https://www.madelonhulsebos.com/assets/pdf/neurips_position_paper.pdf, online preprint (2025).

[13] T. Klein, J. Hoffart, Foundation models for tabular data within systemic contexts need grounding, arXiv preprint arXiv:2505.19825 (2025).

[14] S. Kurashkin, V. Tynchenko, A. S. Borodulin, V. Nelyub, N. O. Kalutsky, T. Connie, The Current Generation of Tabular Foundation Models: A Critical Review, Machine Learning and Knowledge Extraction 8 (8) (2026) 244. doi:10.3390/make8080244.

[15] W. Kong, A. Das, Introducing TabFM: A Zero-Shot Foundation Model for Tabular Data, https://research.google/blog/introducing-tabfm-a-zero-shot-foundation-model-for-tabular-data/, google Research blogpost (2026).
URL https://research.google/blog/introducing-tabfm-a-zero-shot-foundation-model-for-tabular-data/

[16] Synthefy, P.-h. Li, A. Narayanan, S. S. Narasimhan, R. Mallampalli, A. Agrawal, B. Ajan, R. Shah, S. Agarwal, Nori: A Tabular Foundation Model Trained on Synthetic Data, 10.5281/zenodo.20710462, zenodo repository (2026). doi:10.5281/zenodo.20710462.
URL https://doi.org/10.5281/zenodo.20710462

[17] G. Blayer, M. J. Kim, F. Lefebvre, L. Purucker, A. Arazi, E. Shapira, R. Reichart, F. Hutter, M. L. Morvan, D. Holzmüller, G. Varoquaux, STRABLE: Benchmarking Tabular Machine Learning with Strings, arXiv preprint arXiv:2605.12292 (2026).

[18] B. B. Guta, Localized TabICLv2: Scaling Tabular In-Context Learning through k-NN, arXiv preprint arXiv:2608.16429 (2026).

[19] G. Schindler, M. Schambach, J. Höhne, Enhancing Tabular Learners with Context-Aware Semantic Embeddings, arXiv preprint arXiv:2608.03565 (2026).

[20] J. R. Landis, G. G. Koch, The measurement of observer agreement for categorical data, Biometrics (1977) 159–174.

[21] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, I. Polosukhin, Attention is all you need, Advances in Neural Information Processing Systems 30 (2017).

[22] F. d. Breejen, S. Bae, S. Cha, S.-Y. Yun, Fine-tuned in-context learning transformers are excellent tabular data classifiers, arXiv preprint arXiv:2405.13396 (2024).

[23] J. Ma, V. Thomas, R. Hosseinzadeh, A. Labach, J. Cresswell, K. Golestan, G. Yu, A. L. Caterini, M. Volkovs, TabDPT: Scaling tabular foundation models on real data, Advances in Neural Information Processing Systems 38 (2025) 172692–172722.

[24] R. Hosseinzadeh, A. Labach, Z. Xue, S. Han, V. Thomas, A. L. Caterini, TabDPT-Turbo: Efficient In-Context Learning for Tabular Prediction, arXiv preprint arXiv:2608.01400 (2026).

[25] Y. Zeng, T. Dinh, W. Kang, A. C. Mueller, TabFlex: Scaling Tabular Learning to Millions with Linear Attention, in: A. Singh, M. Fazel, D. Hsu, S. Lacoste-Julien, F. Berkenkamp, T. Maharaj, K. Wagstaff, J. Zhu (Eds.), Proceedings of the 42nd International Conference on Machine Learning, Vol. 267 of Proceedings of Machine Learning Research, PMLR, 2025, pp. 74051–74079.

[26] S.-Y. Liu, H.-J. Ye, TabSwift: An Efficient Tabular Foundation Model with Row-Wise Attention, in: Proceedings of the 43rd International Conference on Machine Learning, 2026.
URL https://openreview.net/forum?id=ottLtChHp2

[27] A. Garg, M. Ali, N. Hollmann, L. Purucker, S. Müller, F. Hutter, Real-TabPFN: Improving tabular foundation models via continued pre-training with real-world data, arXiv preprint arXiv:2507.03971 (2025).

[28] L. Grinsztajn, K. Flöge, O. Key, F. Birkel, P. Jund, B. Roof, B. Jäger, D. Safaric, S. Alessi, A. Hayler, M. Manium, R. Yu, F. Jablonski, S. B. Hoo, A. Garg, J. Robertson, M. Bühler, V. Moroshan, L. Purucker, C. Cornu, L. C. Wehrhahn, A. Bonetto, B. Schölkopf, S. Gambhir, N. Hollmann, F. Hutter, TabPFN-2.5: Advancing the state of the art in tabular foundation models, arXiv preprint arXiv:2511.08667 (2025).

[29] M. Spinaci, M. Polewczyk, M. Schambach, S. Thelin, ContextTab: A semantics-aware tabular in-context learner, Advances in Neural Information Processing Systems 38 (2025) 147005–147040.

[30] X. Zhang, D. C. Maddix, J. Yin, N. Erickson, A. F. Ansari, B. Han, S. Zhang, L. Akoglu, C. Faloutsos, M. W. Mahoney, C. Hu, H. Rangwala, G. Karypis, B. Wang, Mitra: Mixed synthetic priors for enhancing tabular foundation models, Advances in Neural Information Processing Systems 38 (2025) 15795–15840.

[31] Y. Tao, X. Zhang, X. Liu, B. Han, D. Maddix, H. Fang, Z. Han, J. Gai, X. Liu, M. Bohlke-Schneider, Yuyang, Wang, G. Friedland, K. Mah, C. v. d. Lee, C. Kong, Mitra-v2 Technical Report, arXiv preprint arXiv:2609.04540 (2026).

[32] X. Zhang, G. Ren, H. Yu, H. Yuan, H. Wang, J. Li, J. Wu, L. Mo, L. Mao, M. Hao, N. Dai, R. Xu, S. Li, T. Zhang, Y. He, Y. Wang, Y. Zhang, Z. Xu, D. Li, F. Gao, H. Zou, J. Liu, J. Liu, J. Xu, K. Cheng, K. Li, L. Zhou, Q. Li, S. Fan, X. Lin, X. Han, X. Li, Y. Lu, Y. Xue, Y. Jiang, Z. Wang, Z. Wang, P. Cui, LimiX: Unleashing structured-data modeling capability for generalist intelligence, arXiv preprint arXiv:2509.03505 (2025).

[33] Y. Wang, X. Zhang, H. Yu, M. Hao, G. Ren, H. Yuan, L. Mao, Y. Zhang, C. Yuan, P. Cui, LimiX-2M: Mitigating Low-Rank Collapse and Attention Bottlenecks in Tabular Foundation Models, in: Proceedings of the 43rd International Conference on Machine Learning, 2026.
URL https://openreview.net/forum?id=YjYPYg112A

[34] X. Zhang, G. Ren, H. Yuan, H. Zou, H. Tan, H. Wang, J. Song, J. Li, J. Zhang, J. Zhang, K. Li, L. Mo, L. Mao, M. Hao, N. Xu, R. Ding, R. Zhang, S. Li, S. Mei, T. Zhang, W. Mu, Y. Dong, Y. Wei, Y. Xue, Y. Wang, Y. He, Z. Yang, Z. Li, D. Li, F. Wang, J. Liu, J. Chen, J. Du, K. Cheng, K. Li, L. Sun, L. Zhou, N. Dai, Q. Wang, R. Xu, S. Du, S. Yang, W. Lu, W. Chu, X. Huang, X. Lin, X. Ai, X. Han, X. Li, X. Su, X. Zhang, Y. Lu, Y. Zhang, Y. Qin, Y. Huang, Y. Xu, Y. Lv, Y. Jiang, Y. Han, P. Cui, LimiX-2: A Contextual Mechanism Network Towards General Structured-Data Intelligence, arXiv preprint arXiv:2609.17488 (2026).

[35] M. Eo, M.-K. Suh, H.-S. Cho, J. Kim, S. Kim, S. Nam, S. Lee, EXAONE Tabular 1.0: Technical Report, arXiv preprint arXiv:2608.25774 (2026).

[36] J. Qu, D. Holzmüller, G. Varoquaux, M. L. Morvan, TabICL: a tabular foundation model for in-context learning on large data, in: Proceedings of the 42nd International Conference on Machine Learning, ICML'25, JMLR.org, 2025.

[37] J. Qu, D. Holzmüller, G. Varoquaux, M. Le Morvan, TabICLv2: A Better, Faster, Scalable, and Open Tabular Foundation Model, in: Proceedings of the 43rd International Conference on Machine Learning, 2026.
URL https://openreview.net/forum?id=SxsyLjIfWB

[38] L. Grinsztajn, K. Flöge, O. Key, F. Birkel, P. Jund, B. Roof, M. Manium, S. B. Hoo, M. Bühler, A. Garg, D. Safaric, J. Robertson, B. Jäger, S. Alessi, A. Hayler, V. Moroshan, L. Purucker, P. Singer, A. Arazi, J. Siems, J. H. Metzen, G. Grab, N. Erickson, S. Guo, E. Kalfon, S. Bing, D. Salinas, C. Cornu, L. C. Wehrhahn, D. Kriuchkova, K. Kaya, L. Sidhoum, M. Salmon, J. Chen, M. Hulsebos, Y. LeCun, S. Müller, B. Schölkopf, S. Gambhir, N. Hollmann, F. Hutter, TabPFN-3: Technical report, arXiv preprint arXiv:2605.13986 (2026).

[39] B. Jäger, N. Erickson, L. Grinsztajn, F. Birkel, K. Flöge, O. Key, K. Kaya, J. Kübler, A. Frankel, T. Schröder, A. Garg, J. H. Metzen, D. Salinas, S. Bing, K. Collins, T. Çelik, V. Balazadeh, L. Sidhoum, T. Pereda, B. Roof, A. Tschalzev, S. Guo, P. Singer, L. Purucker, J. Robertson, M. Salmon, P. Jund, J. Chen, D. Kriuchkova, A. Cahu, E. Kalfon, A. Hayler, G. Grab, V. Monteiro, L. Wehrhahn, D. Safaric, C. Cornu, A. Arazi, R. Grace, S. Alessi, M. Manium, B. Schölkopf, Y. LeCun, M. Hulsebos, S. Gambhir, N. Hollmann, F. Hutter, TabPFN-3.5: Technical Report, arXiv preprint arXiv:2609.17895 (2026).

[40] M. Bouadi, P. Seth, A. Tanna, V. K. Sankarapu, Orion-Bix: Bi-axial attention for tabular in-context learning, in: Proceedings of the ACM Web Conference 2026, 2026, pp. 8673–8676.

[41] M. Bouadi, P. Seth, A. Tanna, V. K. Sankarapu, Orion-MSP: Multi-scale sparse attention for tabular in-context learning, arXiv preprint arXiv:2511.02818 (2025).

[42] P. Pfeiffer, D. Gordeev, M. Müller, L. Fink, J. S. Soler, M. Landry, B. Murray, M. V. Conde, S. S. Ambati, TabH2O: A Unified Foundation Model for Tabular Prediction, arXiv preprint arXiv:2605.18383 (2026).

[43] P. Wang, W. Liu, H. Wang, C. Huang, Y. Sun, Z. Wang, H. Huang, Q. Wang, Z. Xin, P. Hou, J. Yu, C. Liu, E. Meng, B. Wang, Xiaomi-TabLDM: A Tabular Foundation Model Technical Report, arXiv preprint arXiv:2609.03880 (2026).

[44] M. Cho, M. Jeong, D. Lee, J. Lee, J. Yoo, Causilo Technical Report, arXiv preprint arXiv:2609.22866 (2026).

[45] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, BERT: Pre-training of deep bidirectional transformers for language understanding, in: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers), 2019, pp. 4171–4186.

[46] B. Zhu, X. Shi, N. Erickson, M. Li, G. Karypis, M. Shoaran, XTab: Cross-table Pretraining for Tabular Transformers, in: A. Krause, E. Brunskill, K. Cho, B. Engelhardt, S. Sabato, J. Scarlett (Eds.), Proceedings of the 40th International Conference on Machine Learning, Vol. 202 of Proceedings of Machine Learning Research, PMLR, 2023, pp. 43181–43204.

URL https://proceedings.mlr.press/v202/zhu23k.html
[47] Y. Yang, Y. Wang, G. Liu, L. Wu, Q. Liu, UniTabE: A universal pretraining protocol for tabular foundation model in data science, in: International Conference on Learning Representations, 2024, pp. 27862–27877.
[48] M. Spinaci, M. Polewczyk, J. Hoffart, M. C. Kohler, S. Thelin, T. Klein, PORTAL: Scalable tabular foundation models via content-specific tokenization, arXiv preprint arXiv:2410.13516 (2024).
[49] J. Yan, B. Zheng, H. Xu, Y. Zhu, D. Chen, J. Sun, J. Wu, J. Chen, Making pre-trained language models great on tabular prediction, in: International Conference on Learning Representations, 2024, pp. 23570–23593.
[50] A. Arazi, E. Shapira, R. Reichart, TabSTAR: A tabular foundation model for tabular data with text fields, Advances in Neural Information Processing Systems 38 (2025) 172108–172161.
[51] M. J. Kim, L. Grinsztajn, G. Varoquaux, CARTE: Pretraining and Transfer for Tabular Learning, in: R. Salakhutdinov, Z. Kolter, K. Heller, A. Weller, N. Oliver, J. Scarlett, F. Berkenkamp (Eds.), Proceedings of the 41st International Conference on Machine Learning, Vol. 235 of Proceedings of Machine Learning Research, PMLR, 2024, pp. 23843–23866.
[52] F. Mahdisoltani, J. Biega, F. Suchanek, YAGO3: A knowledge base from multilingual wikipedias, in: 7th Biennial Conference on Innovative Data Systems Research, CIDR Conference, 2014.
[53] M. J. Kim, F. Lefebvre, G. Brison, A. Perez-Lebel, G. Varoquaux, Table Foundation Models: on knowledge pre-training for tabular learning, Transactions on Machine Learning Research (2025).
URL https://openreview.net/forum?id=QV4P8Csw17
[54] S. Hegselmann, A. Buendia, H. Lang, M. Agrawal, X. Jiang, D. Sontag, TabLLM: Few-shot classification of tabular data with large language models, in: International Conference on Artificial Intelligence and Statistics, PMLR, 2023, pp. 5549–5581.
[55] X. Wen, H. Zhang, S. Zheng, W. Xu, J. Bian, From supervised to generative: A novel paradigm for tabular deep learning with large language models, in: Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2024, pp. 3323–3333.
[56] J. Gardner, J. C. Perdomo, L. Schmidt, Large scale transfer learning for tabular data via language modeling, Advances in Neural Information Processing Systems 37 (2024) 45155–45205.
[57] G. Schindler, M. Schambach, M. Medek, S. Thelin, TabGemma: Text-Based Tabular ICL via LLM using Continued Pretraining and Retrieval, arXiv preprint arXiv:2511.03570 (2025).
[58] H. Dong, P. Zhang, M. Lu, Y. Shen, G. Ke, MachineLearningLM: Scaling Many-shot In-context Learning via Continued Pretraining, arXiv preprint arXiv:2509.06806 (2025).
[59] A. Müller, C. Curino, R. Ramakrishnan, MotherNet: Fast training and inference via hyper-network transformers, in: International Conference on Learning Representations, 2025, pp. 76666–76686.
[60] D. Bonet, D. M. Montserrat, X. Giró-i Nieto, A. G. Ioannidis, HyperFast: Instant classification for tabular data, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 38, 2024, pp. 11114–11123.
[61] D. Bonet, M. Comajoan Cara, A. Calafell, D. Mas Montserrat, A. G. Ioannidis, iLTM: Integrated large tabular model, in: Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Volume 2, 2026, pp. 186–197.
[62] M. Bouadi, N. Bouarour, V. Kulkarni, S. Dubey, A. Tanna, V. K. Sankarapu, Shaping the Prior: How Synthetic Task Distributions Determine Tabular Foundation Model Quality, arXiv preprint arXiv:2605.18971 (2026).
[63] G. Eggert, K. Huo, M. Biven, J. Waugh, TabLib: A dataset of 627m tables with context, arXiv preprint arXiv:2310.07875 (2023).
[64] F. M. Suchanek, M. Alam, T. Bonald, L. Chen, P.-H. Paris, J. Soria, YAGO 4.5: A large and clean knowledge base with a rich taxonomy, in: Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval, 2024, pp. 131–140.
[65] A. Pfefferle, J. Hog, L. Purucker, F. Hutter, nanoTabPFN: A lightweight and educational reimplementation of TabPFN, arXiv preprint arXiv:2511.03634 (2025).
[66] Y. Wu, D. L. Bergman, Zero-shot Meta-learning for Tabular Prediction Tasks with Adversarially Pre-trained Transformer, in: A. Singh, M. Fazel, D. Hsu, S. Lacoste-Julien, F. Berkenkamp, T. Maharaj, K. Wagstaff, J. Zhu (Eds.), Proceedings of the 42nd International Conference on Machine Learning, Vol. 267 of Proceedings of Machine Learning Research, PMLR, 2025, pp. 67111–67127.
[67] M. Peroni, F. Le, V. Sheinin, Robust Tabular Foundation Models, arXiv preprint arXiv:2512.03307 (2025).
[68] A. O. Davies, T. de Menezes e Silva Filho, N. Ajmeri, Mind the Gap? A Distributional Comparison of Real and Synthetic Priors for Tabular Foundation Models, arXiv preprint arXiv:2605.06343 (2026).
[69] Z. Türkmen, K. Kaya, A. Pfefferle, F. Hutter, Towards Evaluating Data Priors for Tabular Foundation Models, arXiv preprint arXiv:2606.29241 (2026).
[70] S.-Y. Liu, H.-R. Cai, Q.-L. Zhou, H.-H. Yin, T. Zhou, J.-P. Jiang, H.-J. Ye, TALENT: A Tabular Analytics and Learning Toolbox, Journal of Machine Learning Research 26 (226) (2025) 1–16.
URL http://jmlr.org/papers/v26/25-0512.html
[71] H. Zhao, R. Thompson, D. Steinberg, A. Rahman, E. V. Bonilla, C. S. Ong, From Synthetic Priors to Model Behavior: Structural Coverage in Tabular Foundation Models, arXiv preprint arXiv:2609.06912 (2026).
[72] L. Regenwetter, R. Yu, C. Picard, F. Ahmed, Engineering Regression Without Real-Data Training: Domain Adaptation for Tabular Foundation Models Using Multi-Dataset Embeddings, arXiv preprint arXiv:2603.04692 (2026).
[73] K. Helli, D. Schnurr, N. Hollmann, S. Müller, F. Hutter, Drift-resilient TabPFN: In-context learning temporal distribution shifts on tabular data, Advances in Neural Information Processing Systems 37 (2024) 98742–98781.
[74] Z. Guan, Y. Chen, H. Qian, K. Han, P. Cui, F. Wu, K. Kuang, StablePFN: Stable Prediction with Causal-Aware Tabular Foundation Model, in: Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Volume 2, 2026, pp. 1381–1392.
[75] X. Lv, Y. Mao, R. Xu, C. Zheng, Y. Chen, H. Li, J. Yang, Y. Chen, K. Kuang, M. Geng, S. Liu, W. Huang, S. Yang, W. Yang, Z. Lin, H. Wang, When Tabular Foundation Models Meet Strategic Tabular Data: A Prior Alignment Approach, in: Proceedings of the 43rd International Conference on Machine Learning, 2026.
URL https://openreview.net/forum?id=jlQeiq1azU

[76] M. Bühler, L. Purucker, F. Hutter, Causal data augmentation for robust fine-tuning of tabular foundation models, arXiv preprint arXiv:2601.04110 (2026).
[77] L. Berti-Equille, Prior-Aligned Data Cleaning for Tabular Foundation Models, arXiv preprint arXiv:2604.25154 (2026).
[78] M. Biloš, J. T. Wilson, A. Schneider, Y. Nevmyvaka, A Mechanistic Study of Tabular Foundation Models, arXiv preprint arXiv:2605.21288 (2026).
[79] M. Arbel, D. Salinas, F. Hutter, EquitabPFN: A target-permutation equivariant prior fitted network, Advances in Neural Information Processing Systems 38 (2025) 62586–62609.
[80] A. R. Balef, M. Koshil, K. Eggensperger, Towards Understanding Layer Contributions in Tabular In-Context Learning Models, arXiv preprint arXiv:2511.15432 (2025).
[81] A. R. Balef, M. Koshil, K. Eggensperger, Is One Layer Enough? Understanding Inference Dynamics in Tabular Foundation Models, in: Proceedings of the 43rd International Conference on Machine Learning, 2026.
URL https://openreview.net/forum?id=IxpVGcg39J
[82] R. Knauer, E. Rodner, In Search of Grandmother Cells: Tracing Interpretable Neurons in Tabular Representations, arXiv preprint arXiv:2601.03657 (2026).
[83] R. Knauer, A. Beinrucker, E. Rodner, ConceptTracer: Interactive Analysis of Concept Saliency and Selectivity in Neural Representations, arXiv preprint arXiv:2604.07019 (2026).
[84] A. Gupta, A. Sethi, D. Kumar, TabPFN Through The Looking Glass: An interpretability study of TabPFN and its internal representations, arXiv preprint arXiv:2601.08181 (2026).
[85] H.-J. Ye, S.-Y. Liu, W.-L. H. Chao, A closer look at TabPFN v2: Understanding its strengths and extending its capabilities, Advances in Neural Information Processing Systems 38 (2025) 135605–135637.
[86] F. Hoppe, L. Kleinemeier, A. Franz, U. Göbel, Comparing Task-Agnostic Embedding Models for Tabular Data, arXiv preprint arXiv:2511.14276 (2025).
[87] C. McCarter, What exactly has TabPFN learned to do?, arXiv preprint arXiv:2502.08978 (2025).
[88] J. Zheng, C. Gordon, Y. Ji, H. Saratchandran, S. Lucey, From Tables to Signals: Revealing Spectral Adaptivity in TabPFN, arXiv preprint arXiv:2511.18278 (2025).
[89] K. Sharma, S. Tiwari, O. Nema, P. Pareek, Mechanistic Evidence for Spectral Structures in Prior-Data Fitted Networks, arXiv preprint arXiv:2601.21731 (2026).
[90] W. Tenachi, Y. Hezaveh, L. Perreault-Levasseur, P.-L. Bacon, Do Tabular Foundation Models Know Physics? Contamination, Units, and the Deterministic Limit, arXiv preprint arXiv:2609.02766 (2026).
[91] T. Nagler, Statistical foundations of prior-data fitted networks, in: Proceedings of the 40th International Conference on Machine Learning, PMLR, 2023, pp. 25660–25676.
[92] N. Shaheen, J. Ma, A. Labach, F. Hutter, V. Thomas, A. L. Caterini, Understanding the Surprising Generalization Properties of Tabular Foundation Models, arXiv preprint arXiv:2608.17957 (2026).
[93] J. Hu, M. Ghelichi, Noise Immunity in In-Context Tabular Learning: An Empirical Robustness Analysis of TabPFN's Attention Mechanisms, arXiv preprint arXiv:2604.04868 (2026).
[94] A. Gupta, D. Kumar, M. Mandal, S. Deshpande, Where Computation Lives Inside TabPFN: Causal Localisation of Attention Head Function, arXiv preprint arXiv:2606.12917 (2026).
[95] A. Tanna, Y. Desai, P. Seth, M. Bouadi, N. Bouarour, V. K. Sankarapu, Ensembling Tabular Foundation Models - A Diversity Ceiling And A Calibration Trap, arXiv preprint arXiv:2605.18696 (2026).
[96] J. L. d. M. Costa, F. Popineau, A. Rimmel, B.-L. Doan, High Performance, Low Reliability: Uncertainty Benchmarking for Tabular Foundation Models, in: 34th European Symposium on Artificial Neural Networks, Computational Intelligence and Machine Learning (ESANN 2026), Bruges, Belgium, 2026.
[97] C. Klötergens, V. K. Yalavarthi, L. Schmidt-Thieme, T. Hanika, Do Tabular Foundation Models Agree with Themselves?, arXiv preprint arXiv:2608.06004 (2026).
[98] J. Landsgesell, P. Knoll, T. Wenzel, Distributional regression with tabular foundation models: Evaluating probabilistic predictions via proper scoring rules, arXiv preprint arXiv:2603.08206 (2026).
[99] T. R. Johnson, K. Ben-Jacob, N. Negarandeh, O. Vendrell-Gallart, R. Bostanabad, On the Uncertainty Quantification Ability of Tabular Foundation Models, Computing in Science & Engineering (2026).
[100] T. Nagler, D. Rügamer, Uncertainty Quantification for Prior-Data Fitted Networks using Martingale Posteriors, arXiv preprint arXiv:2505.11325 (2025).
[101] S. Fortini, K. Ng, S. Petrone, J. Rousseau, S. Wei, Uncertainty Decomposition for Bayes-Filtered Transformers via Bayesian Predictive Inference, arXiv preprint arXiv:2602.04596 (2026).
[102] M. Loza, D. Chushig-Muzo, E. Milara, L. Bote-Curiel, L. Estrada-Petrocelli, F. Grijalva, Empirical Evaluation of Out-Of-Distribution Performance of Tabular Foundation Models, arXiv preprint arXiv:2607.26000 (2026).
[103] Z.-J. Cheng, Z.-Y. Jia, Z. Zhou, Y.-F. Li, L.-Z. Guo, Realistic evaluation of TabPFN v2 in open environments, arXiv preprint arXiv:2505.16226 (2025).
[104] L. Purucker, A. Tschalzev, N. Erickson, G. Blayer, D. Holzmüller, A. Arazi, A. Pfefferle, M. Tajjar, G. Varoquaux, F. Hutter, Beyond IID: How General Are Tabular Foundation Models, Really?, arXiv preprint arXiv:2606.30410 (2026).
[105] M. Herre, A. Tschalzev, S. Marton, C. Bartelt, Explaining Tabular Foundation Model Differences Through Meta-Features, arXiv preprint arXiv:2605.28418 (2026).
[106] S. Lee, Mitigating Label Shift in Tabular In-Context Learning via Test-Time Posterior Adjustment, in: Proceedings of the 43rd International Conference on Machine Learning, 2026.
URL https://openreview.net/forum?id=8N2kUoknvy

[107] S. McDowell, N. Stromberg, L. Sankar, Correcting Class Imbalance in Prior-Data Fitted Networks for Tabular Classification, arXiv preprint arXiv:2605.21742 (2026).
[108] P. Kenfack, S. E. Kahou, U. Aïvodji, Towards Fair In-Context Learning with Tabular Foundation Models, Transactions on Machine Learning Research (2026).
URL https://openreview.net/forum?id=AsBhwD0sqo
[109] P. Kenfack, J. C. Cresswell, A. L. Caterini, S. E. Kahou, U. Aïvodji, Training Fair Tabular Foundation Models, arXiv preprint arXiv:2608.14211 (2026).
[110] F. Capano, J. Böhler, Probing Memorization of Tabular In-Context Learning, arXiv preprint arXiv:2606.31208 (2026).
[111] A. Mueller, J. Siems, H. Nori, D. Salinas, A. Zela, R. Caruana, F. Hutter, GAMformer: Bridging tabular foundation models and interpretable machine learning, arXiv preprint arXiv:2410.04560 (2024).
[112] J.-Y. Jia, C. Singh, R. Caruana, B. J. Lengerich, Selecting Feature Interactions for Generalized Additive Models by Distilling Foundation Models, arXiv preprint arXiv:2604.13332 (2026).
[113] R. Miftachov, B. Charron, S. Valentin, Interpretable Tabular Foundation Models via In-Context Kernel Regression, arXiv preprint arXiv:2602.02162 (2026).
[114] D. Rundel, J. Kobialka, C. von Crailsheim, M. Feurer, T. Nagler, D. Rügamer, Interpretable machine learning for TabPFN, in: World Conference on Explainable Artificial Intelligence, Springer, 2024, pp. 465–476.
[115] L. B. T. R. Sena, F. G. Azevedo, Real-Time Explanations for Tabular Foundation Models, arXiv preprint arXiv:2603.29946 (2026).
[116] J. Fonseca, J. Stoyanovich, ExplainerPFN: Towards tabular foundation models for model-free zero-shot feature importance estimations, arXiv preprint arXiv:2601.23068 (2026).
[117] B. Feuer, C. Hegde, N. Cohen, Scaling TabPFN: Sketching and feature selection for tabular prior-data fitted networks, arXiv preprint arXiv:2311.10609 (2023).
[118] M. Abdullah, Understanding Context Sampling in TabPFN on Small Tabular Datasets, arXiv preprint arXiv:2607.26628 (2026).
[119] B. Feuer, R. T. Schirrmeister, V. Cherepanova, C. Hegde, F. Hutter, M. Goldblum, N. Cohen, C. White, TuneTables: Context optimization for scalable prior-data fitted networks, Advances in Neural Information Processing Systems 37 (2024) 83430–83464.
[120] J. Ma, V. Thomas, G. Yu, A. Caterini, In-context data distillation with TabPFN, arXiv preprint arXiv:2402.06971 (2024).
[121] Y. Chen, X. Ding, L. Akoglu, VIP-COP: Context Optimization for Tabular Foundation Models, arXiv preprint arXiv:2605.12904 (2026).
[122] V. Thomas, J. Ma, R. Hosseinzadeh, K. Golestan, G. Yu, M. Volkovs, A. Caterini, Retrieval & fine-tuning for in-context tabular models, Advances in Neural Information Processing Systems 37 (2024) 108439–108467.
[123] S. Jamalidinan, Y. Xu, K. Cheshmi, ARASH: Adaptive Retrieval And Shot Selection for Tabular Prediction, arXiv preprint arXiv:2608.17856 (2026).
[124] J. Heredge, M. J. Villani, P. Deshpande, A. Seshadri, N. Kumar, CRUMB: Efficient Prior Fitted Network Inference via Distributionally Matched Context Batching, arXiv preprint arXiv:2606.11473 (2026).
[125] R. Sergazinov, S.-A. Yin, Chunked TabPFN: Exact training-free in-context learning for long-context tabular data, arXiv preprint arXiv:2509.00326 (2025).
[126] G. Zabërgja, R. Kamel, A. Kadra, C. M. Frey, J. Grabocka, End-to-End Compression for Tabular Foundation Models, in: Proceedings of the 43rd International Conference on Machine Learning, 2026.
URL https://openreview.net/forum?id=84mfkGDxYh
[127] O. Ipas, G. Gomez-Trenado, R. Romero-Zaliz, I. Triguero, LUCoS: Latent Unsupervised Context Selection for Tabular Foundation Models, arXiv preprint arXiv:2605.27254 (2026).
[128] Y. Li, X. Chang, M. Kara, X. Liu, A. Roy-Chowdhury, S. Oymak, When and How Unlabeled Data Provably Improve In-Context Learning, Advances in Neural Information Processing Systems 38 (2025) 31831–31866.
[129] Y. Lin, S. Li, Context-Constrained Transfer Learning for Tabular Foundation Models via Data Distillation, arXiv preprint arXiv:2607.04809 (2026).
[130] D. Nguyen, M. Jawhar, N. Chesneau, TFM-Retouche: A Lightweight Input-Space Adapter for Tabular Foundation Models, arXiv preprint arXiv:2605.06047 (2026).
[131] S.-Y. Liu, H.-J. Ye, TabPFN unleashed: a scalable and effective solution to tabular classification problems, in: Proceedings of the 42nd International Conference on Machine Learning, ICML'25, JMLR.org, 2025.
[132] Y. Shi, C. Liu, A. Yu, Z. Lu, S. Elias, K. Cheng, J. Kou, X. Chen, Y. Liu, H.-Z. Huang, M. Beer, A multi-fidelity tabular prior-data fitted network model for accurate prediction and uncertainty quantification, Nature Communications (2026).
[133] Y. WU, P. Dong, J. Tao, B. Jiang, Do Tabular Foundation Models Still Need Feature Engineering?, arXiv preprint arXiv:2609.13202 (2026).
[134] I. Rubachev, A. Kotelnikov, N. Kartashev, A. Babenko, On finetuning tabular foundation models, arXiv preprint arXiv:2506.08982 (2025).
[135] H. A. Gozeten, M. E. Ildiz, X. Zhang, M. Soltanolkotabi, M. Mondelli, S. Oymak, Test-time training provably improves transformers as in-context learners, in: Proceedings of the 42nd International Conference on Machine Learning, Vol. 267, 2025.
[136] A. Tanna, P. Seth, M. Bouadi, V. K. Sankarapu, TabTune: A unified library for inference and fine-tuning tabular foundation models, in: Companion Proceedings of the ACM Web Conference 2026, 2026, pp. 204–207.
[137] B. Koloski, X. Jiang, S. Pollak, B. Škrlj, M. Jamnik, N. Simidjievski, KnowsTFM: Knowledge-Informed Fine-Tuning of Small Tabular Foundation Models, arXiv preprint arXiv:2606.30258 (2026).
[138] C. Kolberg, J. Kreuer, J. Huurdeman, S. Ouaari, K. Eggensperger, N. Pfeifer, TabPFN-wide: Continued pre-training for extreme feature counts, arXiv preprint arXiv:2510.06162 (2025).
[139] A. Z. S. B. Habib, M. Y. Ahamed, P. K. Gyawali, G. Doretto, D. A. Adjeroh, GOTabPFN: From Feature Ordering to Compact Tokenization for Tabular Foundation Models on High-Dimensional Data, in: Proceedings of the 43rd International Conference on Machine Learning, 2026.
URL https://openreview.net/forum?id=fpqfV3lCIB
[140] K. Lee, MultiTabPFN: Codebook-based extensions of TabPFN for high-class-count tabular classification, Neural Networks (2026) 108932.

[141] C. Cureton, N. Armanfard, TabPFN-MT: A Natively Multitask In-Context Learner for Tabular Data, arXiv preprint arXiv:2605.20234 (2026).
[142] M. Tajjar, A. Pfefferle, L. Purucker, F. Hutter, Towards Pretraining Text Encoders for TabPFN, arXiv preprint arXiv:2606.04876 (2026).
[143] A. Tanna, N. Bouarour, M. Bouadi, V. K. Sankarapu, P. Seth, Pocket Foundation Models: Distilling TFMs into CPU-Ready Gradient-Boosted Trees, arXiv preprint arXiv:2605.18654 (2026).
[144] J. Küken, L. Purucker, F. Hutter, Early stopping tabular in-context learning, arXiv preprint arXiv:2506.21387 (2025).
[145] S. Luo, M. M. Kanaan, C. Gordon, A. Leontjeva, S. Lucey, Memory Efficient Tabular Foundation Models, arXiv preprint arXiv:2607.27546 (2026).
[146] J. M. Kübler, B. Jäger, K. Flöge, N. Hollmann, F. Hutter, Attention Quantization for Tabular Foundation Models, arXiv preprint arXiv:2609.13031 (2026).
[147] M. Koshil, M. Feurer, K. Eggensperger, TACTICL: Task-Aware Compression of Tabular ICL Models, in: International Conference on Automated Machine Learning, 2026.
URL https://openreview.net/forum?id=IgpWPDN81P
[148] A. Sudjianto, A. Zhang, Compositional Layer Pruning of Frozen Tabular Foundation Models: When Single-Layer Importance Misleads, and What to Do About It, SSRN working paper (2026). doi:10.2139/ssrn.6745619.
URL https://ssrn.com/abstract=6745619
[149] F. Koch, M. Wever, F. Raisch, B. Tischler, State-Space Models for Tabular Prior-Data Fitted Networks, arXiv preprint arXiv:2510.14573 (2025).
[150] H. Wang, S. Zhang, W. Ma, SOMTab: Set-Order Mamba for Efficient Tabular In-Context Learning, arXiv preprint arXiv:2608.27882 (2026).
[151] S. B. Ozturk, A. Pfefferle, F. Hutter, Speedrunning Tabular Foundation Model Pretraining, arXiv preprint arXiv:2606.03681 (2026).
[152] C. Dudley, S. Oymak, Latent Chain-of-Thought Improves Structured-Data Transformers, arXiv preprint arXiv:2605.11262 (2026).
[153] B. Bischl, G. Casalicchio, M. Feurer, P. Gijsbers, F. Hutter, M. Lang, R. G. Mantovani, J. N. van Rijn, J. Vanschoren, OpenML Benchmarking Suites, in: Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks, 2021.
URL https://openreview.net/forum?id=OCrD8ycKjG
[154] J. Landsgesell, P. Knoll, T. Wenzel, ScoringBench: A Benchmark for Evaluating Tabular Foundation Models with Proper Scoring Rules, arXiv preprint arXiv:2603.29928 (2026).
[155] M. Mráz, B. Das, A. Gupta, L. Purucker, F. Hutter, Towards benchmarking foundation models for tabular data with text, arXiv preprint arXiv:2507.07829 (2025).
[156] L. Vogel, K. Srinivas, N. D'Souza, S. Shirai, O. Hassanzadeh, H. Samulowitz, Towards Universal Tabular Embeddings: A Benchmark Across Data Tasks, arXiv preprint arXiv:2604.21696 (2026).
[157] J. Kreuer, S. Ouaari, J. Hellmig, J. Braitinger, N. Pfeifer, TabBench-Bio: A Living Benchmark for Machine Learning on High-Dimensional Biomedical Tables, arXiv preprint arXiv:2609.07441 (2026).
[158] D. Jiang, R. Cao, Z. Dang, L. Huang, Q. Zhang, Z. Wang, S. Piao, S. Zhu, J. Chang, Z. Lin, Q. Tian, OmniTabBench: Mapping the Empirical Frontiers of GBDTs, Neural Networks, and Foundation Models for Tabular Data at Scale, arXiv preprint arXiv:2604.06814 (2026).
[159] Y. Wang, Z. Miao, L. Yang, H. Jia, W. Yan, C. Qian, L. Li, TabSieve: Explicit in-table evidence selection for tabular prediction, arXiv preprint arXiv:2602.11700 (2026).
[160] S. B. Hoo, S. Müller, D. Salinas, F. Hutter, From Tables to Time: Extending TabPFN-v2 to Time Series Forecasting, Transactions on Machine Learning Research (2026).
URL https://openreview.net/forum?id=KIkQj8VOUY
[161] S. Dooley, G. S. Khurana, C. Mohapatra, S. V. Naidu, C. White, ForecastPFN: Synthetically-trained zero-shot forecasting, Advances in Neural Information Processing Systems 36 (2023) 2403–2426.
[162] E. O. Taga, M. E. Ildiz, S. Oymak, TimePFN: Effective multivariate time series forecasting with synthetic data, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 39, 2025, pp. 20761–20769.
[163] A. Potapczynski, R. K. Selvam, T. Konstantinova, M. Wolff, K. G. Olivares, R. Ma, M. W. Mahoney, A. G. Wilson, B. N. Oreshkin, D. Efimov, Time-Aware Prior Fitted Networks for Zero-Shot Forecasting with Exogenous Variables, Transactions on Machine Learning Research (2026).
URL https://openreview.net/forum?id=nJARpxp3cF
[164] S. K. Bhethanabhotla, O. Swelam, J. Siems, D. Salinas, F. Hutter, Mamba4Cast: Efficient zero-shot time series forecasting with state space models, arXiv preprint arXiv:2410.09385 (2024).
[165] A. Auer, P. Podest, D. Klotz, S. Böck, G. Klambauer, S. Hochreiter, TiRex: Zero-shot forecasting across long and short horizons with enhanced in-context learning, Advances in Neural Information Processing Systems 38 (2025) 57529–57580.
[166] K. Rasul, A. Ashok, A. R. Williams, H. Ghonia, R. Bhagwatkar, A. Khorasani, M. J. Darvishi Bayazi, G. Adamopoulos, R. Riachi, N. Hassen, M. Biloš, S. Garg, A. Schneider, N. Chapados, A. Drouin, V. Zantedeschi, Y. Nevmyvaka, I. Rish, Lag-Llama: Towards foundation models for time series forecasting, in: R0-FoMo: Robustness of Few-shot and Zero-shot Learning in Large Foundation Models, 2023.
[167] A. Das, W. Kong, R. Sen, Y. Zhou, A decoder-only foundation model for time-series forecasting, in: R. Salakhutdinov, Z. Kolter, K. Heller, A. Weller, N. Oliver, J. Scarlett, F. Berkenkamp (Eds.), Proceedings of the 41st International Conference on Machine Learning, Vol. 235 of Proceedings of Machine Learning Research, PMLR, 2024, pp. 10148–10167.
[168] A. F. Ansari, L. Stella, C. Turkmen, X. Zhang, P. Mercado, H. Shen, O. Shchur, S. S. Rangapuram, S. Pineda Arango, S. Kapoor, J. Zschiegner, D. C. Maddix, H. Wang, M. W. Mahoney, K. Torkkola, A. G. Wilson, M. Bohlke-Schneider, Y. Wang, Chronos: Learning the Language of Time Series, Transactions on Machine Learning Research (2024).
URL https://openreview.net/forum?id=gerNCVqqtR
[169] X. Shi, S. Wang, Y. Nie, D. Li, Z. Ye, Q. Wen, M. Jin, Time-MoE: Billion-scale time series foundation models with mixture of experts, in: International Conference on Learning Representations, 2025, pp. 34635–34667.
[170] Y. Liu, G. Qin, Z. Shi, Z. Chen, C. Yang, X. Huang, J. Wang, M. Long, Sundial: A Family of Highly Capable Time Series Foundation Models, in: A. Singh, M. Fazel, D. Hsu, S. Lacoste-Julien, F. Berkenkamp, T. Maharaj, K. Wagstaff, J. Zhu (Eds.), Proceedings of the 42nd

International Conference on Machine Learning, Vol. 267 of Proceedings of Machine Learning Research, PMLR, 2025, pp. 39295–39317.
[171] M. Jin, S. Wang, L. Ma, Z. Chu, J. Y. Zhang, X. Shi, P.-Y. Chen, Y. Liang, Y.-F. Li, S. Pan, Q. Wen, Time-LLM: Time series forecasting by reprogramming large language models, in: International Conference on Learning Representations, 2024, pp. 23857–23880.
[172] A. F. Ansari, O. Shchur, J. Küken, A. Auer, B. Han, P. Mercado, S. S. Rangapuram, H. Shen, L. Stella, X. Zhang, M. Goswami, S. Kapoor, D. C. Maddix, P. Guerron, T. Hu, J. Yin, N. Erickson, P. M. Desai, H. Wang, H. Rangwala, G. Karypis, Y. Wang, M. Bohlke-Schneider, Chronos-2: From Univariate to Universal Forecasting, arXiv preprint arXiv:2510.15821 (2025).
URL https://arxiv.org/abs/2510.15821
[173] J. Ma, A. Dankar, G. Stein, G. Yu, A. Caterini, TabPFGen – Tabular Data Generation with TabPFN, arXiv preprint arXiv:2406.05216 (2024).
[174] A. Margeloiu, X. Jiang, N. Simidjievski, M. Jamnik, TabEBM: A tabular data augmentation method with distinct class-specific energy-based models, Advances in Neural Information Processing Systems 37 (2024) 72094–72144.
[175] J. Feitelberg, D. Saha, K. Choi, Z. Ahmad, A. Agarwal, R. Dwivedi, TabImpute: Universal Zero-Shot Imputation for Tabular Data, arXiv preprint arXiv:2510.02625 (2025).
[176] X. Lin, C. Xu, M. Yang, G. Cheng, CTSyn: A foundation model for cross tabular data generation, in: International Conference on Learning Representations, 2025, pp. 47938–47959.
[177] X. Jiang, M. Liu, N. Simidjievski, T. Klein, M. Jamnik, Tabular Foundation Model for Generative Modelling, arXiv preprint arXiv:2605.09424 (2026).
[178] D. I. Kim, W. S. Lai, K. W. Zhang, Tabular foundation models can do survival analysis, arXiv preprint arXiv:2601.22259 (2026).
[179] Y. Lyu, S. H. Lin, X. Huang, Z. Li, A censoring-aware target interface for tabular foundation models in survival prediction, arXiv preprint arXiv:2607.09577 (2026).
[180] M. V. Vieyra, Staying Alive: Uncensored Survival Analysis with Tabular Foundation Models, arXiv preprint arXiv:2606.03689 (2026).
[181] D. Seletkov, P. Hager, G. Kaissis, R. Braren, D. Rueckert, R. Rehms, Survival In-Context: Amortized Bayesian Survival Analysis via Prior-Fitted Networks, arXiv preprint arXiv:2603.29475 (2026).
[182] S.-a. Qi, V. Balazadeh, M. Cooper, R. Greiner, R. G. Krishnan, SurvivalPFN: Amortizing Survival Prediction via In-Context Bayesian Inference, arXiv preprint arXiv:2605.15488 (2026).
[183] S. Böhm, L. Purucker, F. Hutter, P. Schlosser, SurvPFN: Towards Foundation Models for Survival Predictions, arXiv preprint arXiv:2606.04564 (2026).
[184] J. Robertson, A. Reuter, S. Guo, N. Hollmann, F. Hutter, B. Schölkopf, Do-PFN: In-context learning for causal effect estimation, Advances in Neural Information Processing Systems 38 (2025) 174811–174848.
[185] Y. Ma, D. Frauen, E. Javurek, S. Feuerriegel, Foundation models for causal inference via prior-data fitted networks, in: International Conference on Learning Representations, 2026, pp. 79065–79098.
[186] J. Robertson, N. Hollmann, S. Müller, N. Awad, F. Hutter, FairPFN: A Tabular Foundation Model for Causal Fairness, in: A. Singh, M. Fazel, D. Hsu, S. Lacoste-Julien, F. Berkenkamp, T. Maharaj, K. Wagstaff, J. Zhu (Eds.), Proceedings of the 42nd International Conference on Machine Learning, Vol. 267 of Proceedings of Machine Learning Research, PMLR, 2025, pp. 51787–51808.
[187] V. Melnychuk, V. Balazadeh, S. Feuerriegel, R. G. Krishnan, Frequentist Consistency of Prior-Data Fitted Networks for Causal Inference, in: Proceedings of the 43rd International Conference on Machine Learning, 2026.
URL https://openreview.net/forum?id=p1GwEz0pix
[188] O. Swelam, L. Purucker, J. Robertson, H. Raum, J. Boedecker, F. Hutter, Does TabPFN Understand Causal Structures?, arXiv preprint arXiv:2511.07236 (2025).
[189] J. Vetter, M. Gloeckler, D. Gedon, J. H. Macke, Effortless, simulation-efficient Bayesian inference using tabular foundation models, Advances in Neural Information Processing Systems 38 (2025) 82685–82724.
[190] P. Bhaskaran, T. Viering, Transformers can do Bayesian Clustering, arXiv preprint arXiv:2510.24318 (2025).
[191] S. Müller, M. Feurer, N. Hollmann, F. Hutter, PFNs4BO: In-context learning for Bayesian optimization, in: Proceedings of the 40th International Conference on Machine Learning, PMLR, 2023, pp. 25444–25470.
[192] R. Yu, C. Picard, F. Ahmed, GIT-BO: High-dimensional Bayesian optimization with tabular foundation models, in: International Conference on Learning Representations, 2026, pp. 111636–111663.
[193] S. Adriaensen, H. Rakotoarison, S. Müller, F. Hutter, Efficient Bayesian learning curve extrapolation using prior-data fitted networks, Advances in Neural Information Processing Systems 36 (2023) 19858–19886.
[194] H. Rakotoarison, S. Adriaensen, N. Mallik, S. Garibov, E. Bergman, F. Hutter, In-Context Freeze-Thaw Bayesian Optimization for Hyperparameter Optimization, in: R. Salakhutdinov, Z. Kolter, K. Heller, A. Weller, N. Oliver, J. Scarlett, F. Berkenkamp (Eds.), Proceedings of the 41st International Conference on Machine Learning, Vol. 235 of Proceedings of Machine Learning Research, PMLR, 2024, pp. 41982–42008.
[195] Y. S. Tan, K. Ng, R. Deng, S. Loganathan, Q. Zhang, B. Chakraborty, PFN-TS: Thompson Sampling for Contextual Bandits via Prior-Data Fitted Networks, arXiv preprint arXiv:2605.10137 (2026).
[196] D. Schiff, O. Lindenbaum, Y. Efroni, ICR-RL: Deep Reinforcement Learning via In-Context-Regression, in: Proceedings of the 43rd International Conference on Machine Learning, 2026.
[197] Q. Liu, Z. Jiang, T. Ayanlade, A. K. Nirala, Y. Li, A. Balu, S. Sarkar, TabQL: In-Context Q-Learning with Tabular Foundation Models, arXiv preprint arXiv:2605.18979 (2026).
[198] D. Eremeev, G. Bazhenov, O. Platonov, A. Babenko, L. Prokhorenkova, Turning tabular foundation models into graph foundation models, arXiv preprint arXiv:2508.20906 (2025).
[199] A. Hayler, X. Huang, I. I. Ceylan, M. Bronstein, B. Finkelshtein, Bringing graphs to the table: Zero-shot node classification via tabular foundation models, arXiv preprint arXiv:2509.07143 (2025).
[200] D. Eremeev, O. Platonov, G. Bazhenov, A. Babenko, L. Prokhorenkova, GraphPFN: A Prior-Data Fitted Graph Foundation Model, in: Proceedings of the 43rd International Conference on Machine Learning, 2026.

URL https://openreview.net/forum?id=340Ep8sfJ3

[201] T. Liao, C. Hu, Y. Sui, X. Zhang, P. Cui, J. Li, Z. Zhang, TFMLinker: Universal link predictor by graph in-context learning with tabular foundation models, arXiv preprint arXiv:2602.08592 (2026).

[202] Y. Gao, J. Bai, H. Huang, Z. Xie, Y. Li, H. T. Tsang, S. Han, Y. Song, KGPFN: Unlocking the Potential of Knowledge Graph Foundation Model via In-Context Learning, arXiv preprint arXiv:2605.14907 (2026).

[203] Y. Zhang, L. Xu, Q. Gan, D. Wipf, M. Wang, RDBLearn: Simple In-Context Prediction Over Relational Databases, arXiv preprint arXiv:2602.18495 (2026).

[204] A. Hayler, K. Flöge, A. Arazi, R. Ranjan, J. Leskovec, F. Birkel, B. Roof, A. Garg, K. Collins, L. Sidhoum, J. Kübler, S. Guo, O. Key, J. H. Metzen, R. Grace, D. Salinas, A. Cahu, S. Bing, B. Jäger, T. Çelik, M. Manium, V. Monteiro, J. Robertson, J. Chen, E. Kalfon, T. Pereda, L. Wehrhahn, D. Safaric, T. Schroeder, G. Grab, D. Kriuchkova, C. Cornu, P. Singer, N. Erickson, V. Balazadeh, M. Salmon, S. Alessi, K. Kaya, P. Jund, L. Grinsztajn, Y. LeCun, B. Schölkopf, M. Hulsebos, L. Purucker, S. Gambhir, F. Hutter, N. Hollmann, Advancing Open and Reproducible Relational Learning: RelArena-α, TabPFN-Rel and RPI, arXiv preprint arXiv:2608.16319 (2026).

[205] Y. Wang, J. You, C. Shi, M. Zhang, Relational In-Context Learning via Synthetic Pre-training with Structural Prior, in: Proceedings of the 43rd International Conference on Machine Learning, 2026.
URL https://openreview.net/forum?id=kiSm94YKor

[206] R. Ranjan, V. Hudovernik, M. Znidar, C. Kanatsoulis, R. Upendra, M. Mohammadi, J. Meyer, T. Palczewski, C. Guestrin, J. Leskovec, Relational transformer: Toward zero-shot foundation models for relational data, in: International Conference on Learning Representations, 2026, pp. 69149–69174.

[207] Y. Wang, X. Wang, Q. Gan, M. Wang, Q. Yang, D. Wipf, M. Zhang, Griffin: towards a graph-centric relational database foundation model, in: Proceedings of the 42nd International Conference on Machine Learning, ICML'25, JMLR.org, 2025.

[208] V. Hudovernik, F. López, V. Kocijan, A. Nitta, J. E. Lenssen, J. Leskovec, M. Fey, KumoRFM-2: Scaling foundation models for relational learning, arXiv preprint arXiv:2604.12596 (2026).

[209] J. P. Walter, S. Agnihotri, M. Keuper, Images as Tables: In-Context Learning with TabPFN for Low-Data Detection of AI-Generated Images, arXiv preprint arXiv:2606.00872 (2026).

[210] J. Zhang, L. Zhong, Z. Zhu, S. Huang, Y. Xu, TabPFN beyond Tabular Data: Calibration and Accuracy on Multimodal Embeddings, arXiv preprint arXiv:2607.11007 (2026).

[211] J. Luo, Y. Yuan, S. Xu, TIME: TabPFN-integrated multimodal engine for robust tabular-image learning, arXiv preprint arXiv:2506.00813 (2025).

[212] W. Kim, C. Song, H. Kim, MultiModalPFN: Extending Prior-Data Fitted Networks for Multimodal Tabular Learning, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2026.
URL https://openreview.net/forum?id=9jbXVRjCtF